ECU Rev 2.0 Design Documentation

Christopher Kalitin

The purpose of this document is to document the design of the ECU rev 2.0 and explain all major circuit elements. This is a low-level technical description of all systems.

Every major circuit on the schematic should be documented here for future reference. Any systems that interact with the rest of the car (eg. startup switch) should have that interface described here in technical detail.

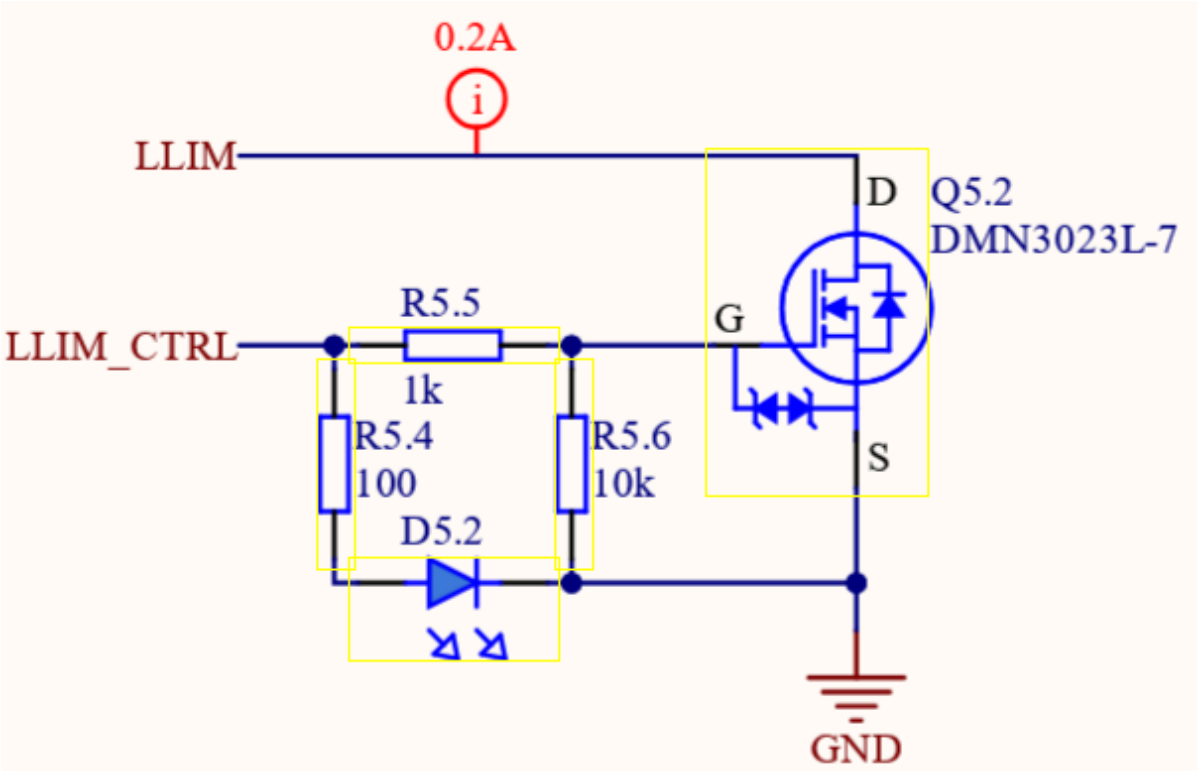
ECU Circuitry Overview

ECU Responsibilities:

1. Contactor Control
2. BMS / Masterboard communication
3. Low Voltage Systems Power
4. Drive Debug LEDs
5. Provide E-Stop 12 V
6. Host DCDC Converter
7. Swap DCDC / Supp
8. Vehicle Startup
9. Discharge / Precharge Motor & MPPTs
10. Hardware Current Faulting
11. Pack Fans Power
12. Main Pack Current Sensing

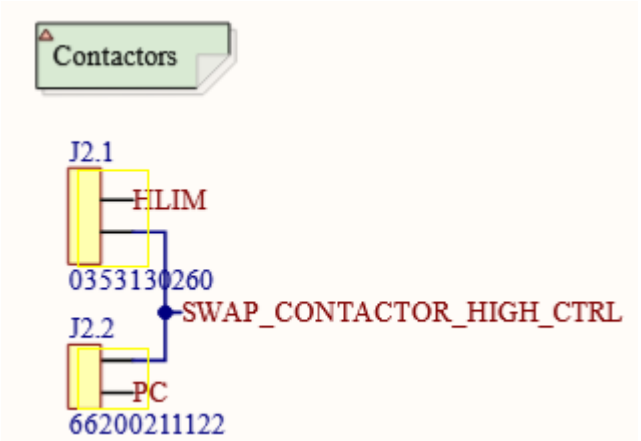
The ECU hosts the finite state machine of the battery. It is responsible for the state of the entire battery and hence it controls power for all other systems in the car.

1. Contactor Control

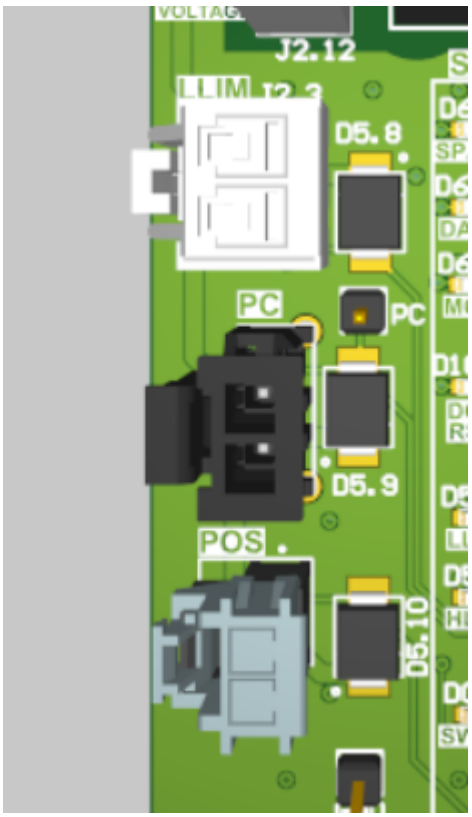


(Figure 1.1) This NFET circuit toggles the ground of the LLIM contactor coil.

(Figure 1.2) This is the flyback diode for the NEG contactor.



(Figure 1.3) This is the schematic for the contactor control wires. Notice that high voltage is provided from SWAP_CONTACTOR_HIGH_CTRL and HLIM / PC are togglable ground s.



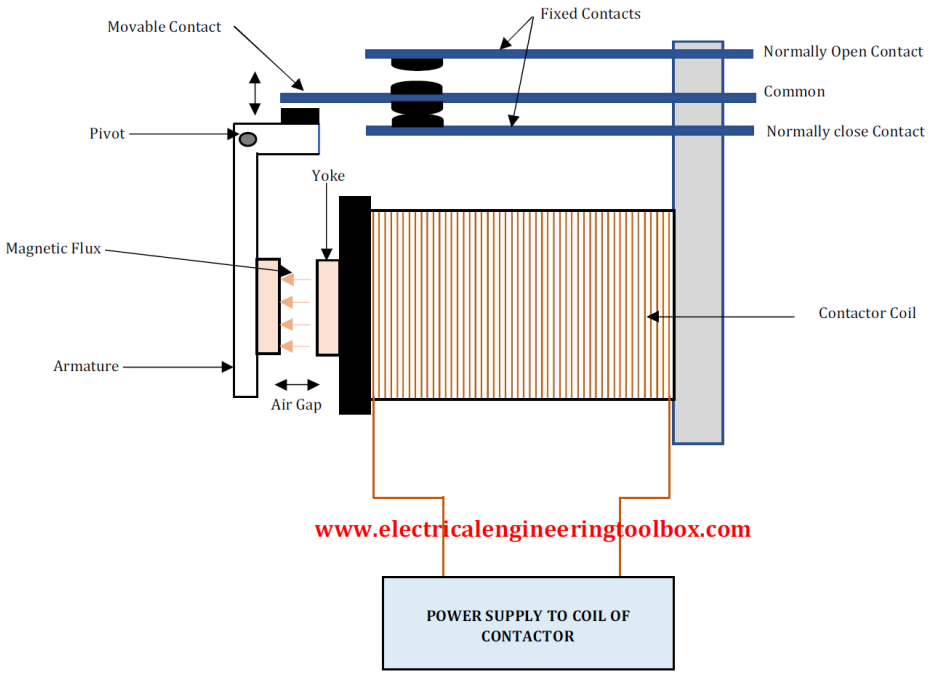
(Figure 1.4) This shows the contactor control connectors on the PCB.

POS, NEG, LLIM Kilovac Contactor: EV200HAANA
HLIM Gigavac Contactor: P115BDA

Note that the Kilovac contactors are rated for 500 A while the Gigavac contactor is rated for 50 A. We expect <10 A of array current, so a smaller contactor is used for HLIM.

1.1 Contactor Control Schematic Overview

In figures 1.1, 1.2, 1.3 you see the contactor control schematics. These circuits toggle the current to the coil of the contactors.



(Figure 1.5) Contactor Internal Diagram

Contactors have two current paths inside them. The primary current path and the coil current path. When current flows through the coils an electromagnetic field is generated which actuates the armature of the contactor. This movement forces a contact between the primary current path of the contactor, closing the switch.

The ECU controls the current going to the coil through low-side switching.

Through the SWAP_CONTACTOR_HIGH_CTRL line (more on this in the startup section), positive 12 V is provided to the contactor coils.

However, the grounds of the coils are initially disconnected. So, there is no current path and the contactors remain open.

When the N-Channel MOSFET associated with a given contactor is closed, the contactor coil negative is connected to the ECU's ground and current flows through the coil.

1.2 Flyback Diode

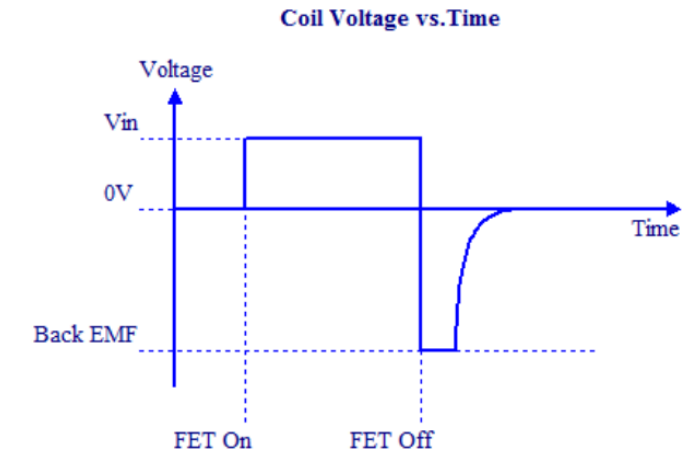
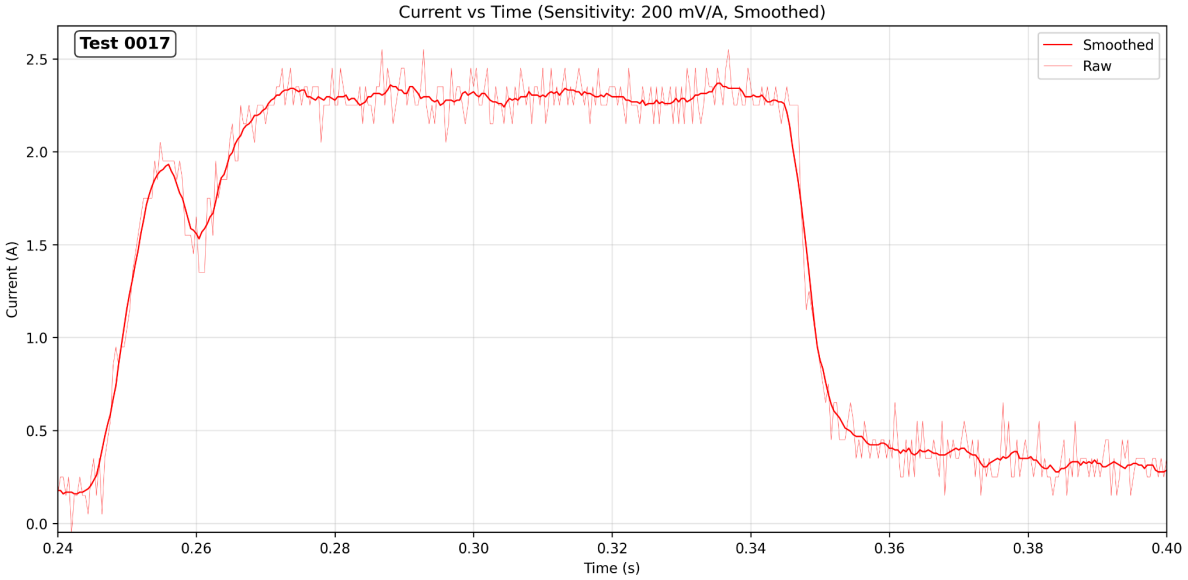


Figure 4. Coil Voltage and Vd vs. Time

(Figure 1.6) Voltage potential over the positive and negative contacts of a contactors coil.

Because all coils act as inductors, we place a flyback diode between the negative of the contactor and SWAP_CONTACTOR_HIGH_CTRL. When the N-Channel MOSFET opens, the inductor attempts to oppose the change in current and builds up a back EMF (a voltage in the opposite direction of previous current flow). To prevent this back EMF from damaging other ECU components, we place the flyback diode so there is an alternative path for current to flow.

1.3 Kilovac Contactor Current Curve



(Figure 1.7) Closing contactor current draw of the kilovac EV200HAANA.

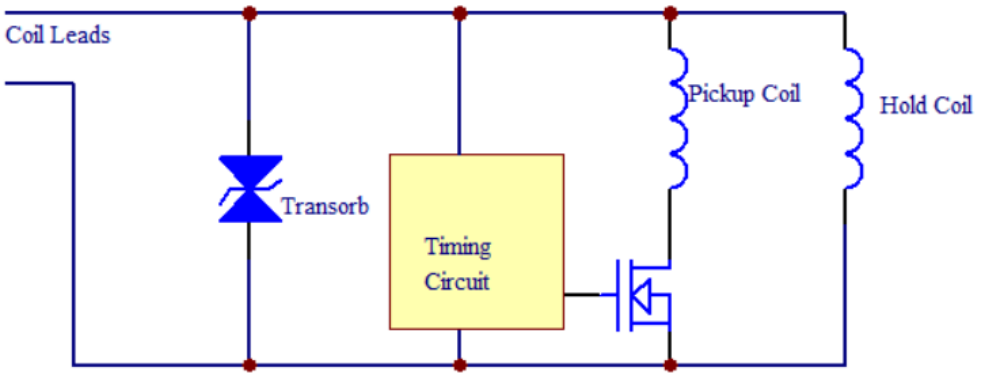


Figure 1. Two Coil Schematic

(Figure 1.8) Internal schematic of a dual coil contactor.

As part of a characterization to find current draw vs. time for the control board, we got data on the closing current and sustained current draw of the contactors. More information about this testing is available in these project updates on Monday and the bottom of this wiki page.

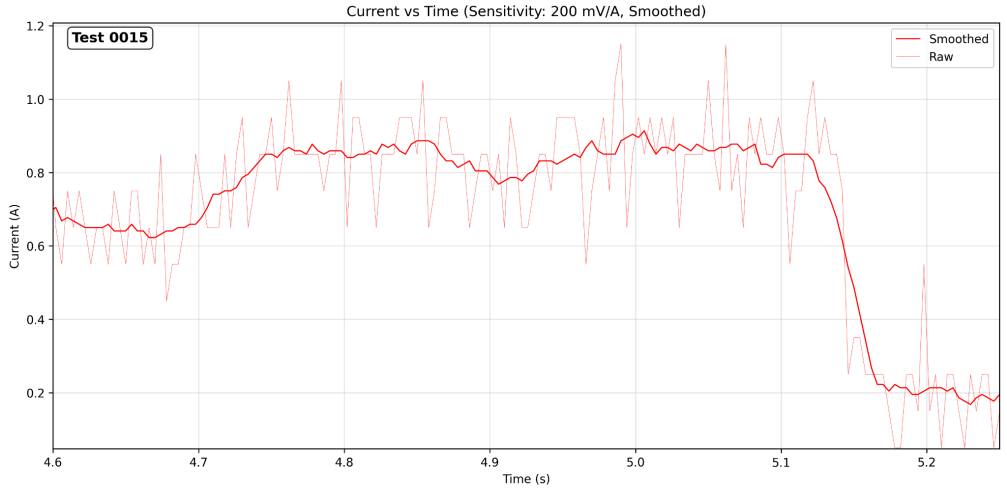
Our Kilovac contactors have two coils that close the armature, a pickup coil and a hold coil. When low voltage is first applied, both coils are active and draw as much current as possible (for a given resistance of the coils, V/R = I) to ensure the contactor closes. After a set interval (130 ms for our contactors), the pickup coil is disabled and only the hold coil remains active. This lowers current draw 10x from ~2 A to ~200 mA.

This explanation describes the initial sustained draw of 2 A and then the reduction to 200 mA, but not the dip to ~1.5 A at 0.26 seconds in figure 1.7.

A contactor is an electromechanical system. At a certain current the electromagnetic force from the coils is enough to overcome the spring force of the armature and it begins moving. Because the armature is a moving piece of metal, it generates a magnetic field which lowers coil current draw. As the armature accelerates the current draw gets lower and lower until the armature impacts the contact and stops moving. At this point current continues increasing up to ~2 A.

Gigavac and Texas Instruments have published great resources to understand this process. Particularly the TI paper.

1.4 Gigavac Contactor Current Curve

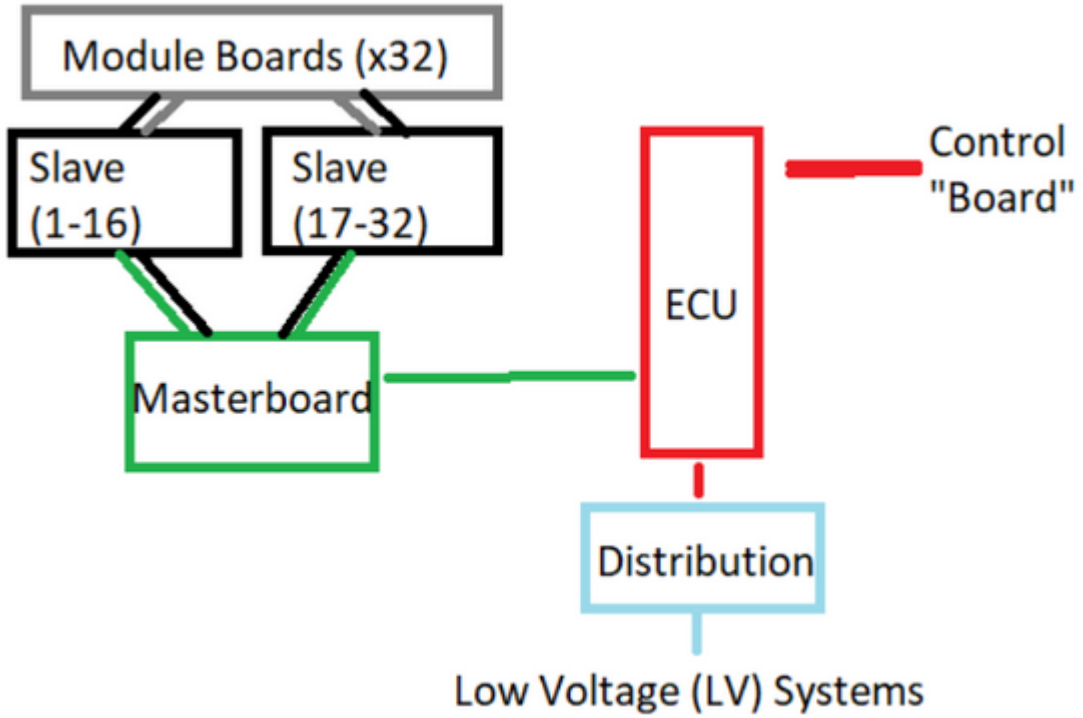


(Figure 1.9) Closing Current of the P115BDA

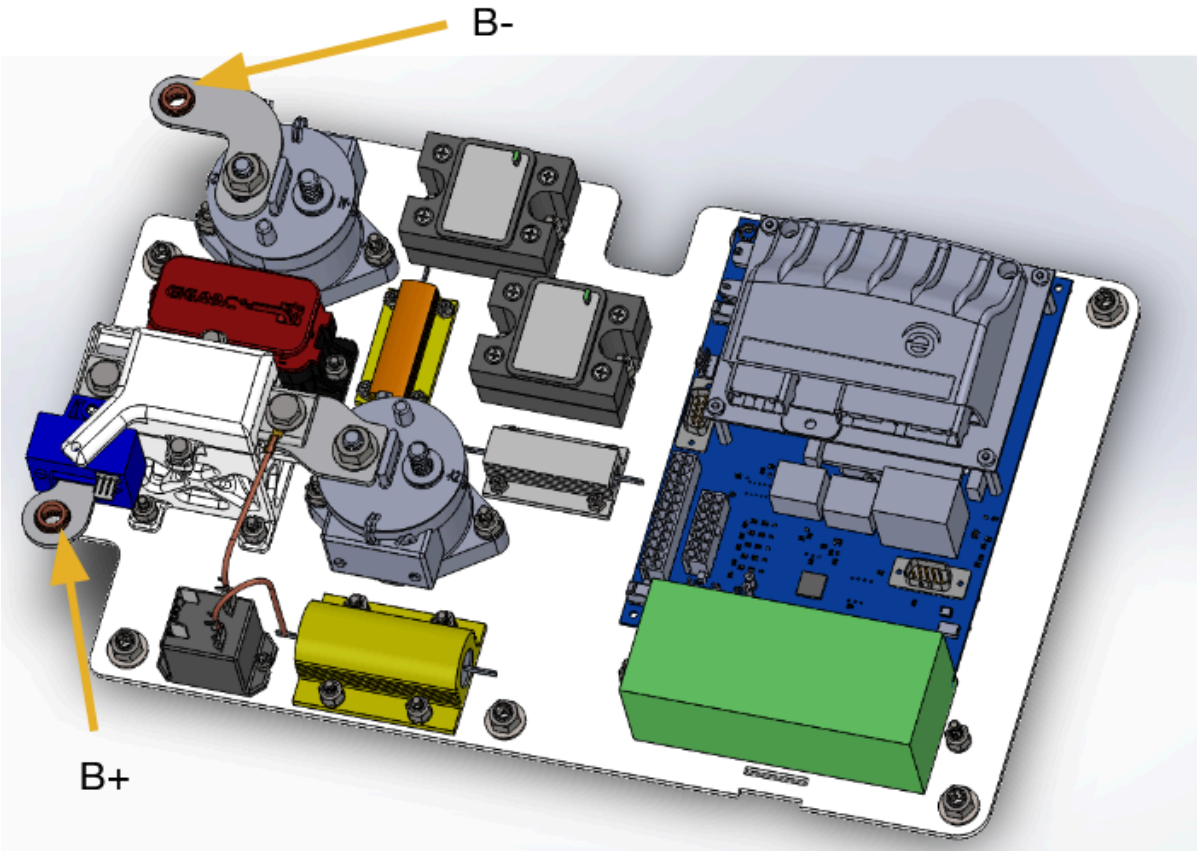
The contactor we use for HLIM does not have a coil economizer, so it draws a constant 200 mA while it's closed. So, in the current chart above we see the contactor start to close and draw ~200 mA above previous current, then at 4.9 seconds the armature begins to move and we see reverse current, and finally the armature closes and current returns to normal.

Note that in the test done to derive the contactor current curve shown above, the control board goes through startup and then faults once it enters the monitoring state. So, at ~5.15 seconds current falls to ~200 mA as all contactors are simultaneously opened.

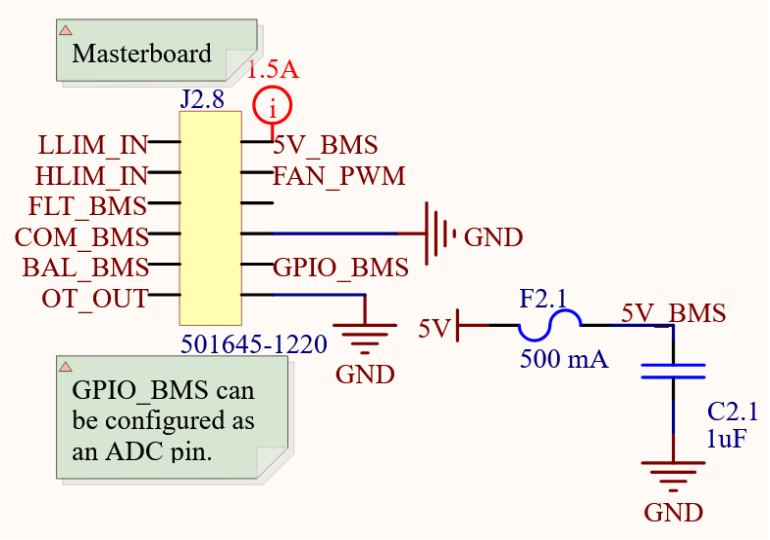
2. BMS / Masterboard Communication



(Figure 2.1) This shows the system topology of our control board PCBs. The module + slave + masterboards make up the BMS while the ECU handles the state and contactors in the battery.



(Figure 2.2) In this previous iteration of the control board, the off-the-shelf Elithion BMS is housed on top of the ECU.



(Figure 2.3) The J2.8 12 pin connector is how masterboard GPIOs are connected to the ECU. The ECU also provides power to the masterboard through a 500 mA SMD fuse.

2.1 BMS Implementation

In a previous iteration of the control board, we used an off-the-shelf Battery Management System (BMS) from the company Elithion.

The Elithion BMS measured the module voltages and temperatures and outputted faults over GPIO pins (eg. there is a dedicated GPIO for a module over voltage fault).

Because this BMS could not control contactors or read current we needed to build a board around it to control the other functions of the pack. This board was named the Elithion Control Unit (ECU), not to be confused with the standard industry term Electronic Control Unit (ECU).

When UBC Solar undertook the process of creating our own in-house BMS, we based the system topology of our system on the existing Elithion BMS. This meant we would reach feature parity with our existing system by replicating voltage readings, temperature readings, and GPIO outputs while maintaining backwards compatibility with the existing Elithion BMS in case our custom solution failed at comp.

We split our system into cell boards that sit on top of each module and have voltage taps to the modules and a thermistor. These voltages are read on slaveboards which send the aggregated data to the masterboard which communicates directly with the ECU. The masterboard uses GPIOs and CAN to communicate with the ECU.

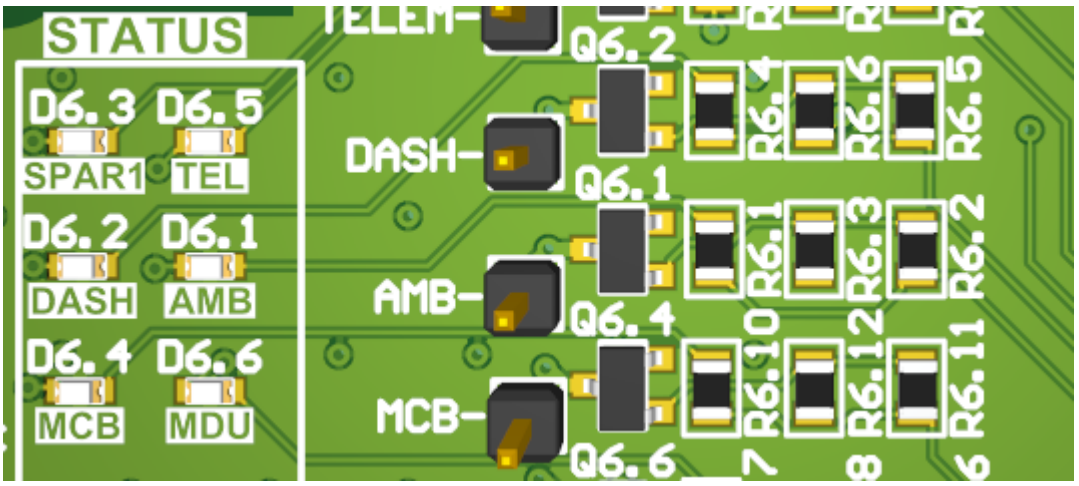
2.2 GPIOs

On the ECU we read 7 GPIOs from the masterboard:

- BAL Balancing active indication
- COM Communication fault (with slave boards) indication
- FLT General Fault indication (active for any fault)
- GPIO General Purpose Input Output - not implemented here; future proofing
- HLIM High limit; highest voltage module is over-voltage
- LLIM Low limit; lowest voltage module is under-voltage
- OT Over temperature indication

Additional BMS data is sent over CAN in messages 0x622 to 0x629 inclusive.

3. Low Voltage Systems Power



(Figure 3.1) A subset of the low-side NFET switching circuits and their respective LEDs.

3.1 Low-side NFET Switching

The current iteration of Brightside has 5 low voltage systems that have their power toggled by the ECU. These are the Driver Display board (DRD), Telemetry board (TEL), Motor Drive Interface (MDI), Maximum Power Point Trackers (MPPTs), and the Memerator (MEM). Other systems get power from the ECU but aren't toggled in the startup sequence such as the driver fans and exterior vehicle lights.

Low-side switching is used to toggle power to these boards. They all have their ow n individual 12V + GND wire harnesses that provide power. While the ground is floating, there is no path for current to flow so the boards are off.

Low-side NFET switching is used for two primary reasons:

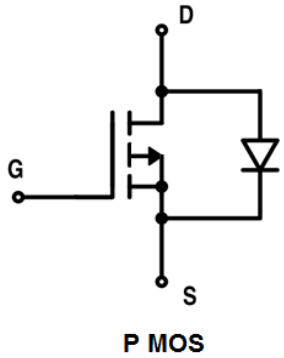
1. N-Channel MOSFETs are easier to drive with a microcontroller GPIO.
2. N-Channel MOSFETs have lower on resistances than P-Channel MOSFETs.

While the gate voltage of an NFET is equal to ground voltage, it is not conducting. Only once gate voltage is brought above the gate-source threshold voltage (Vgs(th)) does it conduct. Vgs(th) for the NFETs on the ECU is 0.8 V (min) to 1.8 V (max).

Vgs(th) is given as a range due to manufacturing tolerances. This range means that some NFETs may conduct if Vgs = 0.8 V and all will conduct at Vgs = 1.8 V. In practice, **we must always design for the maximum Vgs(th) given on the datasheet.**

Due to semiconductor materials science reasons, NFETs have lower on-resistance values (Rds(on)) than PFETs. To illustrate this, the DMN3023L-7 NFETs on the ECU have Rds(on) = 28 mΩ while the DMP2110U-7 PFETs on the distribution board have Rds(on) = 80 mΩ. It is standard industry practice to use NFETs when possible.

3.2 Why not high-side PFET Switching?



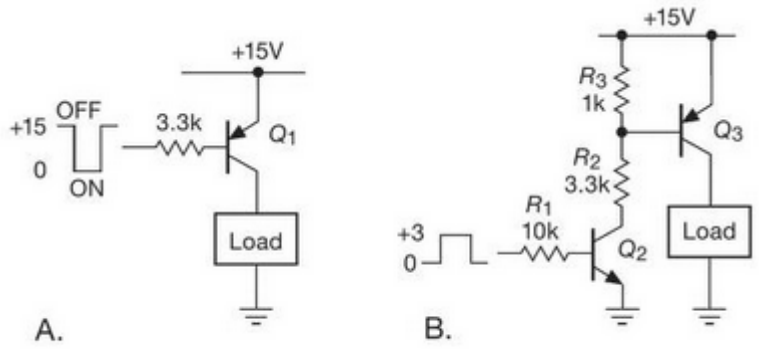
(Figure 3.2) Notice the P-Channel MOSFET's body diode connected from drain to source. This means if drain voltage is 0.7 V (standard diode forward voltage) greater than source voltage, the body diode conducts and current can flow "backwards" through the MOSFET.

If we were to use P-Channel MOSFETs for high-side switching we could toggle 12 V to the boards and keep them grounded normally. Theoretically, this is a safer system because the boards would be de-energized in an off state with no positive voltage source. However, it requires more hardware to switch PFETs from a microcontroller GPIO output than NFETs.

PFETs conduct when gate voltage is less than source voltage by Vgs(th). Ie. Vg < Vs - Vgs(th).

Because all P-Channel MOSFETs have intrinsic body diodes that run from drain to source, we must connect the source of the PFET to the high voltage source (12 V in this case). Otherwise, if drain was connected to 12 V and source was the load, current could always flow through the body diode.

So, to make a PFET with a source voltage (Vs) of 12 V conduct we need to drive gate voltage (Vg) to 12 V - Vgs(th). To illustrate this we will use Vgs(th) = 2 V. So, the PFET will conduct while Vs is less than 10 V. This is great until we want to turn the PFET off and need to find a way to drive Vg to 12 V. Our STM32 microcontrollers can only output 3.3 V so we need another way to let this 3.3 V GPIO output toggle a 12 V trace.



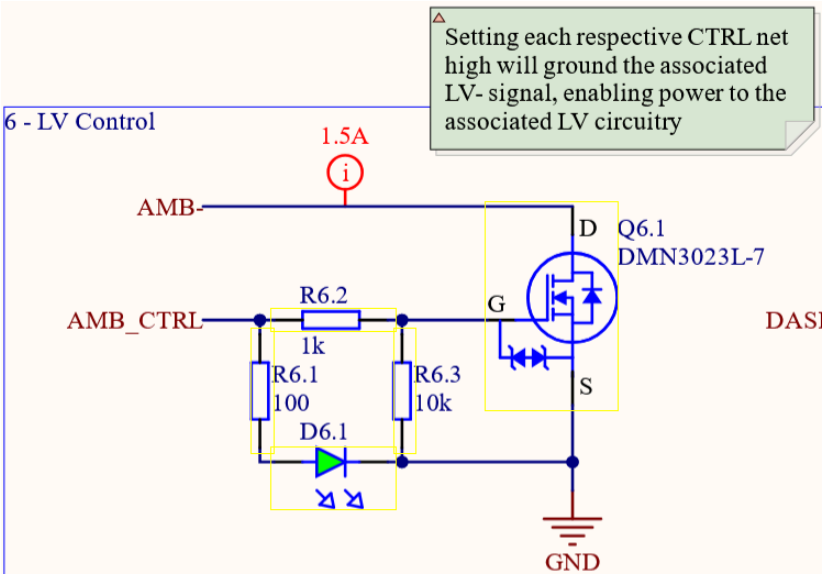
(Figure 3.3) In Circuit B, Q3 is a high-side switching PFET and Q2 is an NFET. This is the circuit required to toggle a PFET with a 3 V microcontroller GPIO output. Source: [The Art of Electronics] section 2.2.2.

In practice, we can change the gate voltage of a PFET by toggling a voltage divider through a low-side NFET.

While the NFET in figure 3.3 circuit B is not conducting, the gate of Q3 is driven high to 15 V. However, when the FET is conducting the voltage divider makes the gate voltage 11.5 V, which is Vgs(th) below 15 V so the PFET conducts.

We could go through all of this effort and add a significant number of components to the ECU, or we can just use low-side NFET switching.

3.3 Switching Circuitry



(Figure 3.4) Low-side NFET switching circuit for the Array Monitoring Board (no longer in the car).

The low-side NFET switches for each low voltage board are made up of 5 components. The NFET, 3 resistors, and an LED.

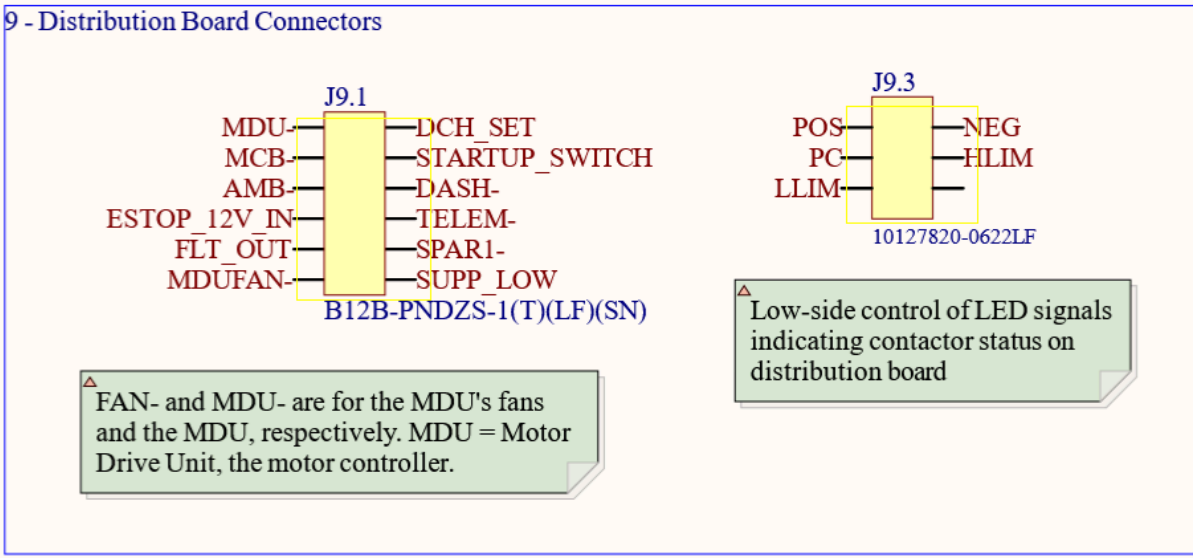
AMB_CTRL is the GPIO output of our microcontroller.
AMB- is the ground of the AMB board.

R6.2 is a current limiting resistor into the gate of the NFET. Because MOSFET gates act as capacitors which are filled with electrons, they produce current spikes if you give them high voltage without a current-limiting resistor.

R6.3 is a pull-down for the gate of the NFET. To ensure the gate is always at a known state even if the GPIO output is floating, we pull it down to GND to ensure the NFET is not conducting.

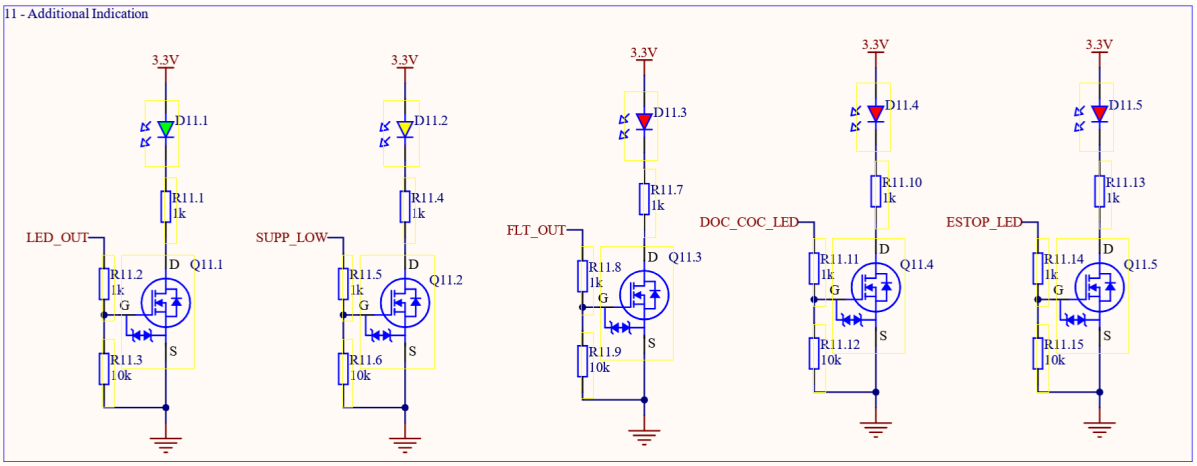
Note that R6.2 and R6.3 make a voltage divider. So, if AMB_CTRL is pulled to 3.3 V, the gate voltage of the NFET is actually ~3V, not 3.3V. Be careful when designing circuits to choose a current-limiting resistor of lower resistance than your pull down resistor. If they were both 10k's, then gate voltage would be ~1.65 V, which may not be enough to switch the NFET.

R6.1 is a current-limiting resistor for the LED D6.1. Since LEDs are diodes and not resistors, they



will take all the current you give them. So, we need to limit current with a resistor so it doesn't act as a short from AMB_CTRL (3.3V) to GND.

4. Drive Debug LEDs



(Figure 4.1) Debug LEDs on the ECU.

(Figure 4.2) LVS power, contactor status, and some debug LEDs signals are sent to the Distribution board through the J9.1 connector.

The LEDs shown in section 3.3 for LVS board power, the LEDs figure 1.1, and some additional debug LEDs are sent to the distribution board so we can know the state of the battery while the top shell of the enclosure is closed.

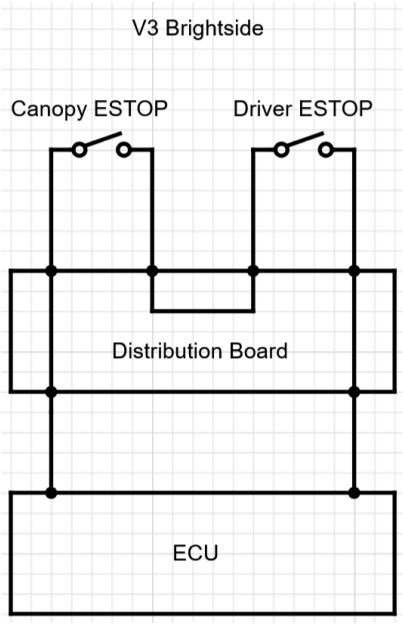
Debug LEDs have very similar MOSFET driver circuits to the low-side switches for LV boards. They differ in that the LED is not just for indication, but the LED is the load.

Because microcontrollers have limited current they can source for GPIOs, we want to limit the current we pull from our STM32s. So, to ensure we don't reach the maximum current our STM32F103RCT6's can output (designed for ~20 mA, could do ~100 mA), we don't directly drive all LEDs. Instead, we toggle a MOSFET that controls the LED. This way power is sourced from the 3.3V trace instead of from the microcontroller.

5. Provide ESTOP 12V

(Figure 5.1) The external ESTOP switch is visible in this photo on the bottom right of the canopy and to the left of Hemat's head with the text "PUSH TO STOP". The driver also has an ESTOP switch.

5.1 ESTOP Topology



(Figure 5.2) ESTOP Topology on the 3rd generation car. Note an ESTOP event occurs if either switch opens. Sign in with battery email to edit.

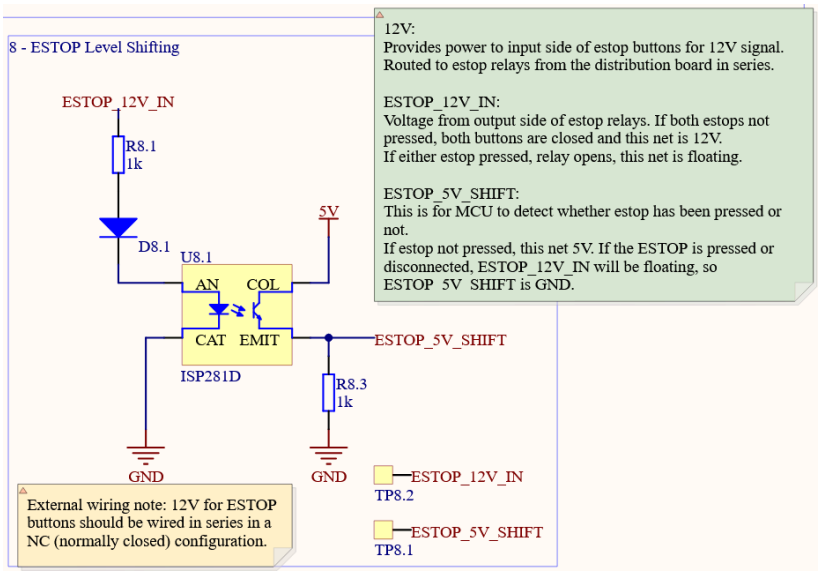
Every emergency stop (ESTOP) connector is normally closed so that the entire line is normally 12 V. 12 V is sourced from the ECU, then goes through two ESTOP

connectors wired in series, then back into the ECU. The ESTOP 12V Input into the ECU is what provides power to the relay that controls the contactors. So, if either ESTOP is pressed, the circuit is broken and all contactors open.

This topology is used so that a faulty state cannot be a valid system state. For example, if one wire is loose and comes out, this counts as an ESTOP and the car stops moving.

If the voltages were swapped and 12 V meant ESTOP was pressed and GND was the normal state, then you could unplug every ESTOP connector (not a valid state) and the ECU would not detect this as invalid.

5.2 Optocoupler



(Figure 5.3) An optocoupler is used on the ECU to toggle a 5V microcontroller input from the 12V ESTOP line. Our STM32 has 5V tolerant pins, but not 12V tolerant pins.

The STM32 microcontroller on the ECU is what reads the state of ESTOP and then outputs its state over the CAN bus.

Since our STM32's GPIO inputs are not rated for 12 V, we toggle a 5 V trace that is read by the STM32. Note that most STM32 GPIO pins can only take 3.3 V and we choose a 5 V tolerant pin for this specific purpose.

To toggle a 5V trace with 12V we use an Optocoupler (also called an Optoisolator). An optocoupler acts as a galvanically isolated switch where a one trace can toggle another without them ever coming into electrical contact. In contrast, a MOSFET gate is always in contact with drain and source through the gate oxide.

Note that we call it an optocoupler and not an optoisolator because this emphasizes the component's ability to couple one signal to another. Optoisolator emphasizes its isolation aspect when this is the primary purpose, eg. isolating high voltage (kV) from low voltage signals.

5.3 Why Not A FET?

We use an optocoupler instead of a FET for three primary reasons:

1. Galvanic Isolation for reading ESTOP
2. Noise immunity
3. Tolerable failure modes

First, the most important reason we use an optocoupler is that it allows for galvanic isolation. An optocoupler has an internal LED that closes a switch on the output side. Photons are the only interaction between both sides so they are galvanically isolated from each other.

Second, the noise immunity of an optocoupler comes from its ability to allow conduction on the output side for a variety of input side currents.

The current capacity of the output side is proportional to the input side current multiplied by the Current Transfer Ratio (CTR). Our ISP281D has a current transfer ratio of 300 to 600%, so 1 mA of current on the input side means the output side could supply 3-6 mA of current.

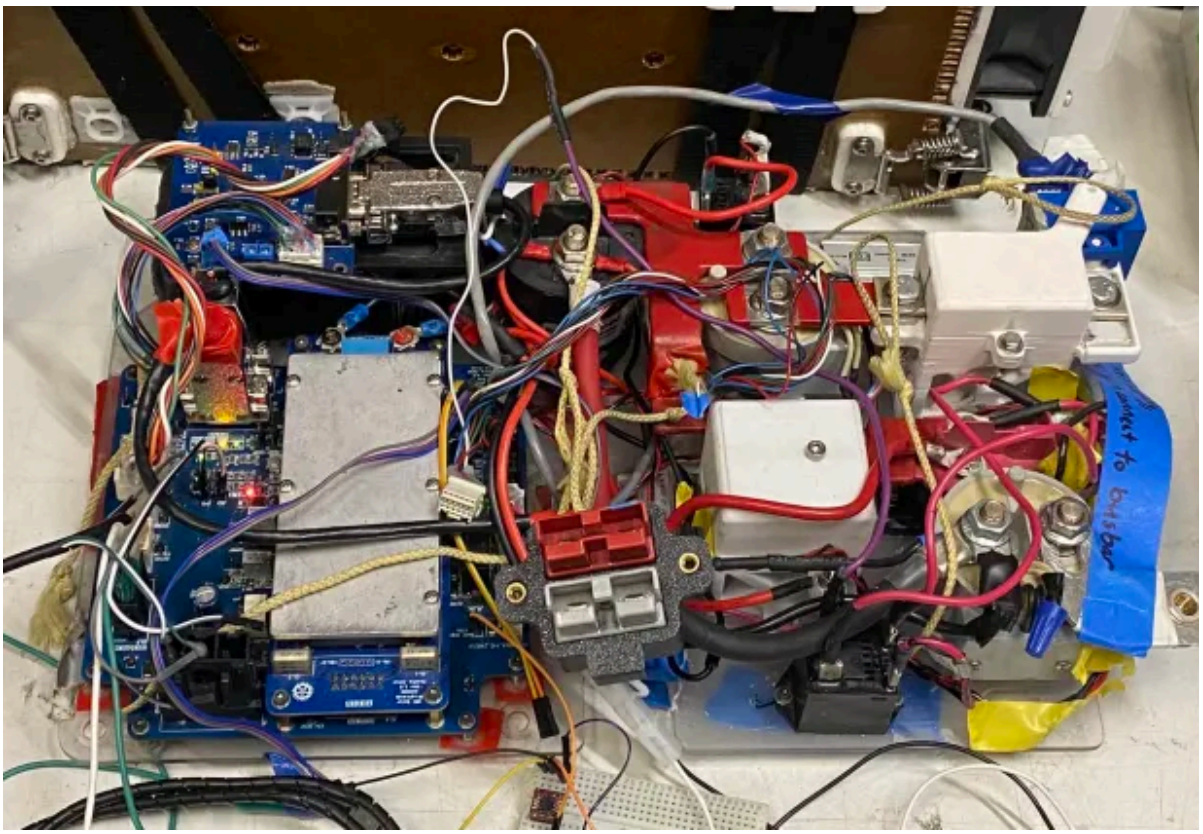
Since the only purpose of the ESTOP_5V_SHIFT trace (output of the optocoupler) is to be read as a GPIO high/low by the microcontroller, current demands are extremely low. For reference, GPIO reads on a microcontroller require microamps of current.

The input side of our optocoupler is 12 V with a 1k current-limiting resistor. By ohm's law, we get 12 mA of current. So, up to 36-72 mA of current could be drawn from the output side of the optocoupler. Even if noise on ESTOP_12V dropped its voltage by 100x to 0.12 V, our microcontroller would still source enough current to read ESTOP_5V_SHIFT as a logic level high.

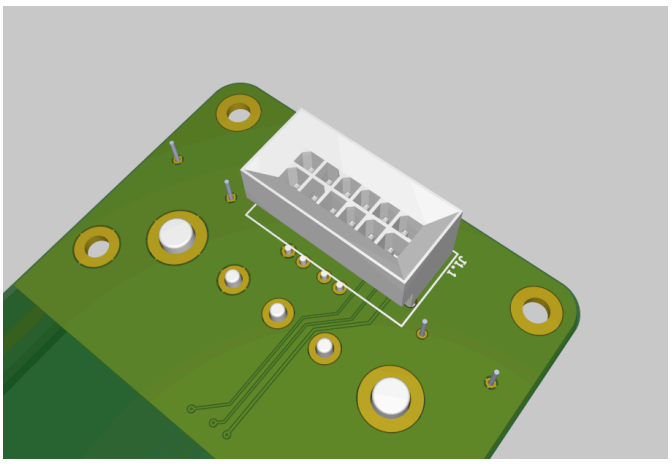
ESTOP is the only case where we use an optoisolator instead of a MOSFET for switching because the ESTOP line is on the order of 10 meters long and goes all along the car. There is ample opportunity for noise to be picked up, especially because it is routed right around the battery which produces significant EMI. Whereas the LVS power control traces are only centimeters long and go directly from an STM32 to a MOSFET on a single PCB.

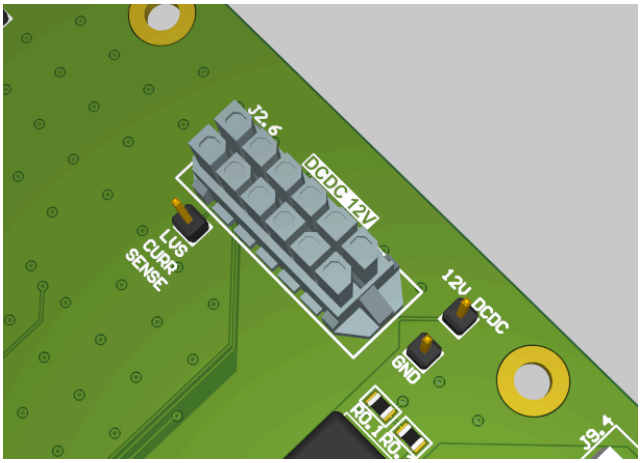
Finally, MOSFETs have failure modes that could cause serious damage to the microcontroller if we use them instead of optocouplers. If, for example, noise on ESTOP_12V gets high enough to exceed the tolerable gate-source voltage of a MOSFET, then the gate oxide layer could break down causing a permanent short between source and gate. This would short 12 V to our microcontroller and likely fry it.

6. Host DCDC Converter

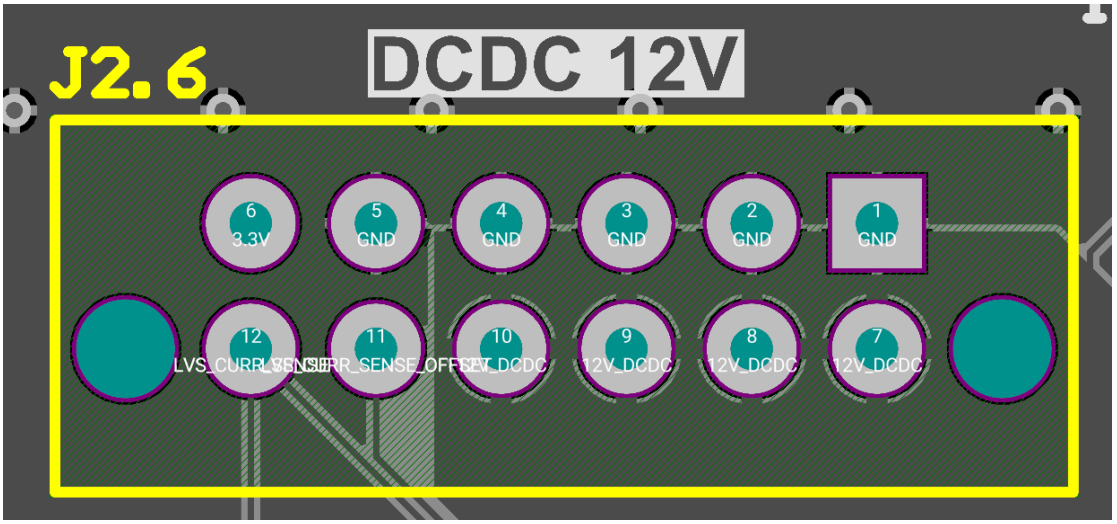


(Figure 6.1) In this image of the control board, the ECU is in the bottom left with the DCDC converter daughterboard and the DCDC's heatsink visible on top of it.



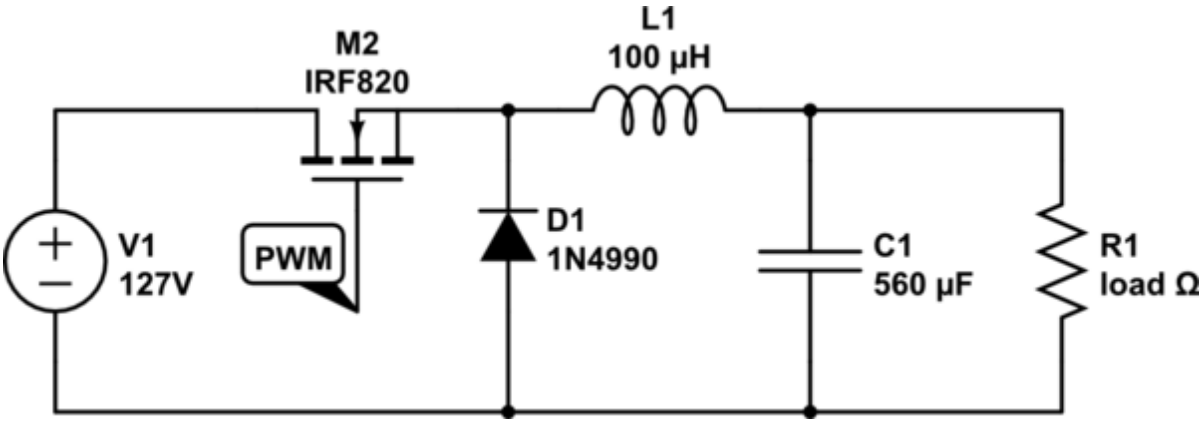


(Figure 6.2) These two images show the DCDC female connector and the male side on the ECU.



(Figure 6.3) This is the pinout of the ECU-side connector. Note pins 12 and 11 are LVS_CURR_SENSE and CURR_SENSE_OFFSET respectively.

6.1 What Is a DCDC?



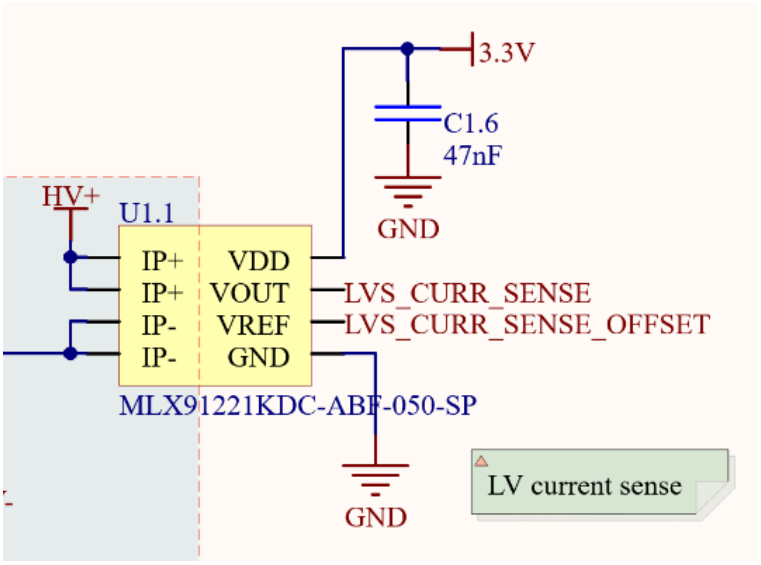
(Figure 6.4) Buck Converter Schematic.

On top of the ECU we have a daughterboard that hosts our V110A12C300BL DCDC converter. The DCDC takes in the main voltage of the battery (134 V max) and outputs 12 V that is used to power all low voltage systems on the car.

Our DCDC is a high-power (300 W) buck converter. At a high level, a buck converter works by periodically charging and discharging an inductor so that its voltage averages out to your desired output voltage.

To illustrate, when we close M2 in Figure 6.4, the inductor acts as a short and begins conducting while it builds up a voltage difference over its terminals. When M2 is opened, the inductor begins discharging and the voltage over its terminals decreases. By timing when we open and close M2 carefully, we can achieve a steady voltage difference over the inductor.

6.2 DCDC Current Sensor



(Figure 6.5) Current sensor on the DCDC.

On the DCDC is a Hall Effect current sensor IC. This current sensor takes in a 3.3V input, creates its own 1.65 V voltage reference (Vref), then outputs a voltage in proportion to the current flowing through it, referenced to the 1.65 V voltage reference. Both the voltage reference and output voltage are read by the ECU's STM32 microcontroller.

The fundamental parameter to understand Hall Effect current sensors is their sensitivity given in millivolts per amp (mV / A).

At zero current draw, a hall effect current sensor will output its voltage reference (1.65 V). When current flows through the current sensor, the output voltage varies in proportion to that current multiplied by its sensitivity.

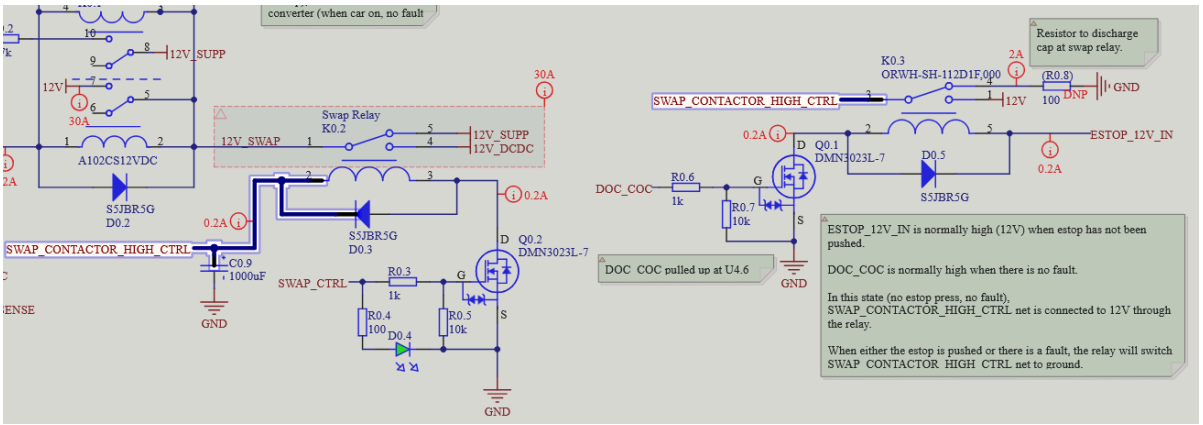
A positive voltage reference allows current sensors to also output negative current and ADC precision degrades when they are near their maximum or minimum voltages.

To illustrate, our [MLX91221]current sensor has a sensitivity of 25 mV / A and a voltage reference of 1.65 ± 0.02 V. If 5 A was flowing through it, its output voltage would be 1.65 + 0.025 * 5 = 1.775 V.

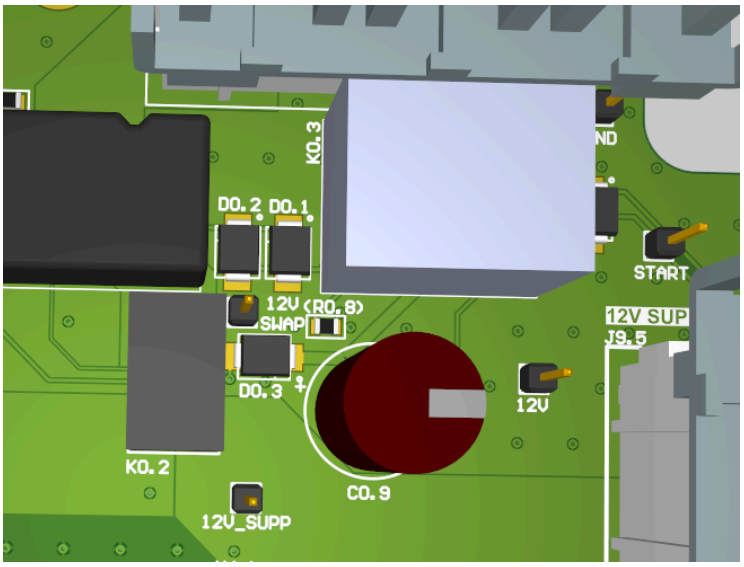
The 12-bit ADCs on our STM32 can read a max voltage of 3.3 V. Dividing 3.3 V by 2^12 we can find the voltage precision of the ADC. 3.3 V / 4096 = 0.8 mV.

Since sensitivity is in units mV/A and our voltage reading is in units mV, we can divide voltage reading precision by sensitivity to find current reading precision. 0.8 mV / 25 mV/A = 0.032 A. So, our DCDC current sensor reading has a precision of 32 mA. A higher sensitivity value would yield a smaller (better) precision value since sensitivity is the denominator.

7. Swap DCDC / SUPP



(Figure 7.1) K0.2 is the DCDC / SUPP Swap relay. K0.3 provides power to the Swap relay's coils only if ESTOP is not pressed.



(Figure 7.2) The Swap relay, Swap Contactor High Control relay, and 1000 uF capacitor are all visible here.

7.1 Swap Relay

We use a EX1-2U1S relay to control whether we draw low voltage power from our supplemental battery or from the DCDC converter. In the battery's initial state, all contactors are open and nothing can draw current from the lithium ion cells, so a supplemental battery is required for initial startup.

Regulations stipulate that we cannot run our entire low voltage system off the supplemental battery during normal driving, so we switch over to our DCDC after startup. See section 8.2.C.1.

When ESTOP is pressed, all battery contactors open so the DCDC will have nowhere to source current from. In this case we have to switch back to the supplemental battery. The default state of the Swap relay is connected to the supplemental battery.

Figure 7.1 shows the Swap relay circuitry. 12 V is provided from the Swap Contactor High Control trace which is only shorted to the ECU's primary 12 V line if ESTOP is low. Otherwise, Swap Contactor High Control is floating and the Swap relay's coils have nowhere to source current from and the relay returns to its default state, connected to the supplemental battery instead of DCDC.

Through this implementation there is hardware that requires ESTOP to not be pressed if we are to source 12V from the DCDC. If ESTOP is pressed the contactors

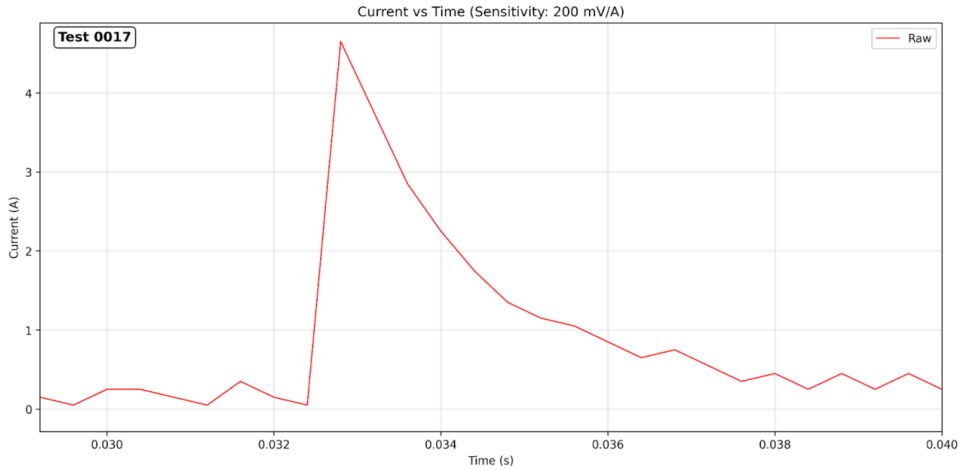
will be open so there will be no way to source current from the DCDC anyway. If the ECU tried to swap to DCDC, its power would cut off as the armature lifts from the supplemental contact and it would power cycle.

Furthermore, if the DOC_COC or charge over current) ever goes high there is no path for current to flow through the Swap trace (Discharge over current Contactor High Control relay. This essentially simulates ESTOP being pressed and is used for hardware current faulting, more on this later.

Both the Swap and Swap Contactor High Control relays have low-side NFET switching circuits to toggle power to the coils of the relays. Both also have flyback diodes to prevent reverse current, as described in section 1.2.

The capacitor C0.9 is where current is sourced from while the armature of the swap contactor is moving. For a moment the ECU cannot source current from either the supplemental battery or the DCDC. The capacitor tides us over in the meantime. Switching time for the EX1-2U1S relay is 2-3 ms.

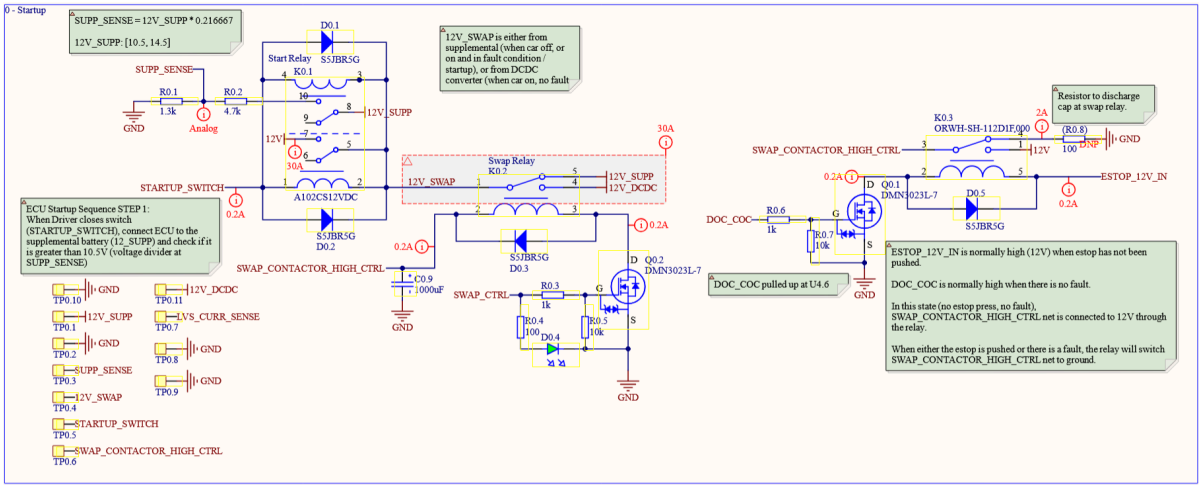
7.2 Capacitor Current Spike



(Figure 7.3) The spike from ~100 mA to >4 A is the 1000 uF capacitor on the ECU charging.

During control board current characterization testing, we found that charging the 1000 uF C0.9 capacitor caused a current spike directly at startup. Since a capacitor with 0 charge acts as a short and there's no current limiting resistor, we get a current spike. More is explained in the attached project update.

8. Vehicle Startup



(Figure 8.2) Startup Circuitry is made up of 3 relays. Each of these with associated components is shown here.

The ECU has 3 relays on it that are related to the startup sequence. These are:

1. Startup Relay
2. Swap & Contactor High Control Relay
3. Swap Relay

8.1 Startup Relay

The startup relay is a Double Pole Double Throw (DPDT) relay. In figure 8.2 the startup relay is the leftmost one. Because it is DPDT, it has two armatures, which both have two contacts (outputs) they can be connected to. The input to both poles is the 12V source, which can either be the supplemental battery or the DCDC Converter.

The default state of both poles is being connected to an output which is floating. While the car is off we want 12 V to be disconnected from everything, and making both poles floating achieves this. Otherwise, if 12 V were connected to the ECU or other LV systems, the quiescence current (current draw when a component is idle) could drain the supplemental battery. To illustrate, if quiescence current is 1mA, you drain 24 mAh per day which would drain our 5 Ah supplemental battery 209 days. The best design practice is to prevent this failure mode by completely disconnecting the supplemental battery.

The poles differ in the purpose of their secondary output. One of the pole second outputs is used to provide 12 V power to the ECU and the rest of the car, while the other pole's second output is used only for voltage sensing of the 12 V line. This allows us to sense supplemental battery voltage while it is the source of 12 V.

Because this is a double pole relay, it also has two coils. The positive of these coils are both connected to the 12 V source (by default this is the supplemental battery). The negative of the coils is connected to the startup switch, which is normally open. So, when you close the startup switch, you connect ground to the negative of the coils and current can flow, closing the startup relay.

8.2 Swap / Contactor High Control Relay

The Swap / Contactor High Control Relay provides 12 V to the contactors and to the coils of the swap relay. If it is open, the contactors cannot close and the swap relay cannot swap to DCDC. It is the rightmost relay in figure 8.2.

The purpose of ESTOP is to isolate the battery from the rest of the car by opening all of the contactors. We have two ways of doing this. First, when the STM32 on the ECU detects that ESTOP has been pressed, all of the GPIOs that control contactor coil power turn off (are pulled to 0 V). Second, we use the Swap / Contactor High Control Relay as a hardware redundancy. It opens when ESTOP is pressed so contactors have no positive voltage source.

The Swap / Contactor High Control Relay opens when ESTOP is pressed because ESTOP is usually pulled to 12 V and is floating when ESTOP is pressed. ESTOP provides positive voltage to the relay's coil, so when it's pressed there is no positive voltage source and the relay opens.

8.3 Swap Relay

The same mechanism of ESTOP controlling the Swap / Contactor High Control Relay is how the Swap / Contactor High Control Relay controls the Swap Relay. The default state of the Swap Relay is being connected to the supplemental battery. When it's closed it connects its output to the DCDC, so DCDC acts as the 12 V source for the car. It can only close when voltage is applied over its coils, and its positive voltage source of the SWAP CONTACTOR HIGH CTRL trace, which is the output of the Swap Contactor High Control Relay. Hence, this trace can only be high while ESTOP is high.

8.4 Overview of Net Voltages

The explanations so far have been all words, so have some tables to get a better understanding of what each vehicle startup event looks like.

8.4.1 Startup Relay States

Net / Trace	Before Startup	After Startup
STARTUP_SWITCH	Floating	GND
12V (ECU)	Floating	12V_Swap

Note that at startup the 12V_Swap relay is connected to the 12_Supp trace. Only after the startup sequence is completed does 12V_Swap become connected to 12V_DCDC.

8.4.2 Swap Relay States

Net / Trace	Trace Type	Default State (car off)	Startup Sequence Completed *	Swap Control Pulled Low **	ESTOP pressed
12V_Swap	Output	12V_Supp	12V_DCDC	12V_Supp	12V_Supp
Swap Contactor High Control	Control 12 V ***	Floating	12V_Swap ****	12V_Swap ****	Floating
Swap Control	Control GPIO ***	Floating **	3.3 V	Floating	3.3 V

* Once the ECU has completed the startup sequence and turned on all LVS boards and closed all relevant contactors, it swaps to DCDC.

** The STM32 microcontroller controls the state of the Swap Control net. When the STM32 is off, it's in a high-impedance state (floating). When the STM32 commands it high, it's pulled to 3.3 V. When the STM32 commands it low, it's pulled to GND.

*** Control traces are what I call traces that toggle power to the relay's coils.

**** Swap Contractor High Control is actually connected to the ECU's 12V plane. This is connected to 12V_Swap through the startup relay.

The primary take away from this table should be that both Swap Contactor High Control and Swap Control must be high (12 V and 3.3 V respectively) for 12V_Swap to be connected to 12V_DCDC.

8.4.3 Swap Contactor High Control

Net / Trace	Trace Type	Default State (car off)	Nominal State (car on)	DOC_COC Pulled Low	ESTOP pressed

Swap Contactor High Control	Output	GND	12V_Swap *	GND	GND
DOC_COC	Control GPIO	Floating	3.3 V	GND	3.3 V
ESTOP 12V IN	Control 12 V	Floating	12 V	12 V	Floating

* Swap Contractor High Control is actually connected to the ECU's 12V plane. This is connected to 12V_Swap through the startup relay.

Note that just like the Swap Relay, the output of the Swap Contactor High Control relay is the logical AND output of DOC_COC and ESTOP_12V_IN. Both must be high (3.3 V and 12 V respectively) for the Swap Contactor High Control output trace to be 12 V.

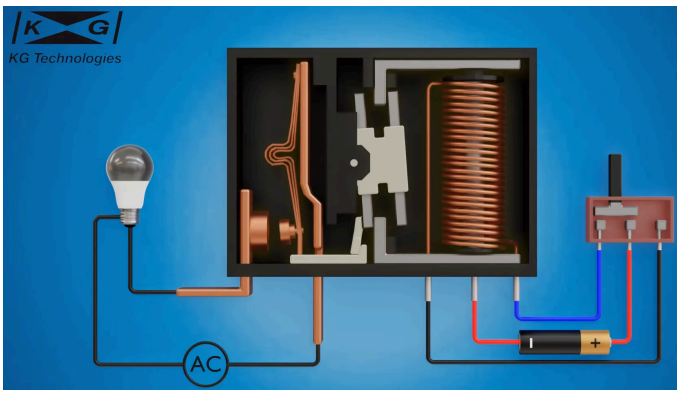
9. Discharge / Precharge Motor & MPPTs

9.1 Why Discharge?

Because the motor controller and motor can maintain residual charge after shutdown of the car, we need to discharge them after the car has been shut down. The motor controller interfaces with the motor for us, so our responsibility is to allow a high-resistance path between motor controller positive and negative so it slowly discharges.

We use a 50 ohm chassis-mount resistor on the Control Board and the ADW1212HTW latching relay for motor controller discharge.

9.2 Latching Relays

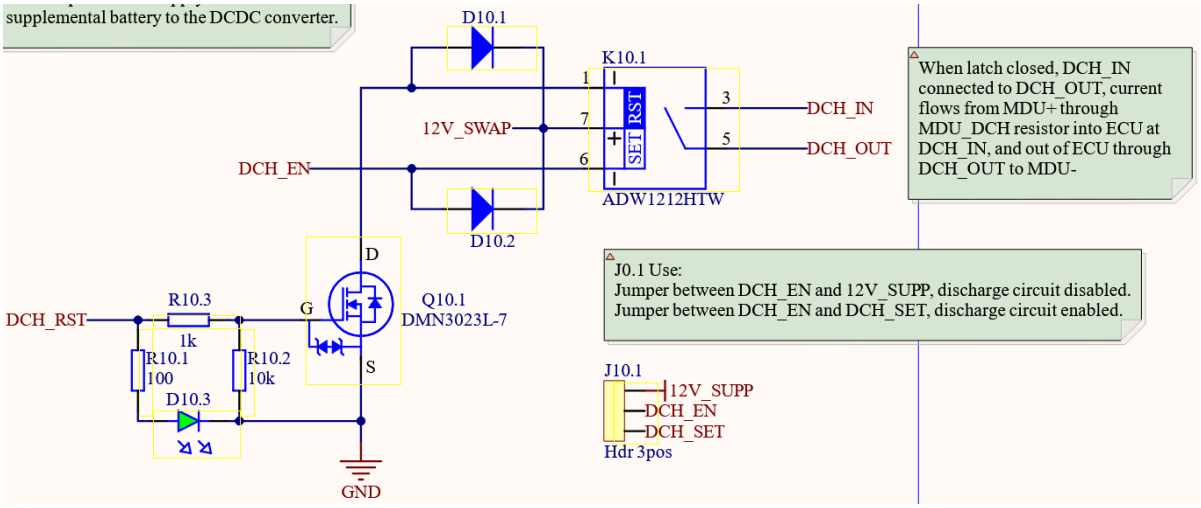


(Figure 9.1) Internals of a latching relay. When a current pulse is applied to the coil, the rotating armature (center) changes state and pushes/pulls the linkage (bottom center left) which is attached to the primary armature (left).

Unlike a non-latching relay which always returns to its default position, a latching relay does not change state unless a current pulse is applied over its coil.

To control the polarity of the coil current pulse (for opening or closing), a latching relay has three coil-side contacts and two coils (note that the coils are wrapped around each other, so it looks like a single coil in figure 9.2). Both coils share a common ground, but when active have opposite polarity. When 12 V is applied to the SET pin, the latching relay closes. When 12 V is applied to the RESET pin, the relay opens.

9.3 Toggling Discharge



(Figure 9.2) Motor Discharge Relay Circuitry. Note that dots at trace intersections signify connections. Ie. pins 1 and 6 are not connected to DCH_EN.

Brightside uses a three position switch (3PST) for the startup switch. Position 1 is the off state of the car while both the STARTUP_SWTICH and DCH_SET lines are floating. Position 2 connected DCH_SET to ground. Position 3 connected STARTUP_SWITCH to ground. So, to startup or shutdown the car, you must go through the intermediary stage of position 2, which DCH_SET is connected to ground.

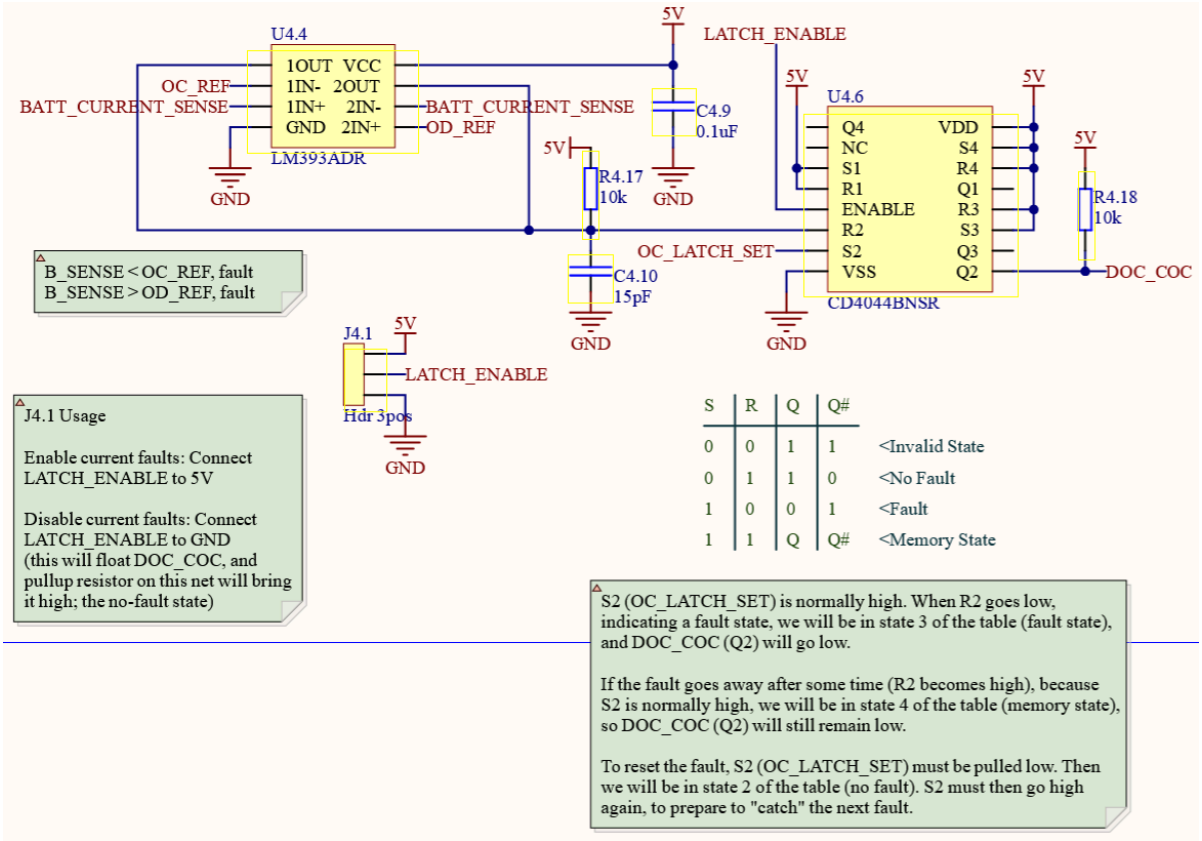
On the ECU, the set pin on the discharge relay is connected to the DCH_EN trace. Through the J10.1 jumper - which offers the ability of disabling the discharge relay - DCH_EN is shorted to DCH_SET. So, for the brief period while the startup switch is in position 2, the set pin of the relay is connected to ground and current flows through the set relay, closing the discharge relay. Hence, the discharge relay is closed every time the car is turned on or off.

Only the microcontroller on the ECU has the ability to open the discharge relay. The DCH_SET trace (controlled by the driver through the startup switch) only closes the relay. When we want to drive the car, the MCU closes a low-side NFET which allows current to flow through the reset relay, opening the relay.

Note that the set / reset time for our ADW1212HTW latching relay is 15 ms, so we are assuming that position 2 of the startup switch is reached for at least 15 ms, otherwise we may not be discharging the motor.

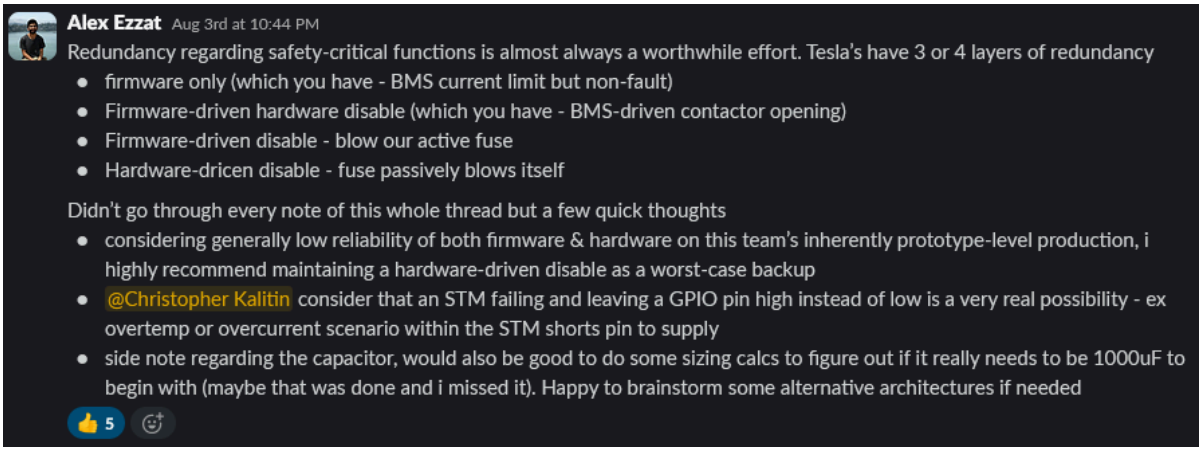
[Why accept 15 ms latching time, Mischa?]

10. Hardware Current Faulting



(Figure 10.1) This shows the full (excluding voltage references) hardware current faulting circuitry. We use a comparator (U4.4) to sense whenever we get an over current or under current fault, and a CMOS NAND Latch (U4.6) to remember faults even after current returns to nominal levels.

10.1 Why Hardware Current Faulting?



(Figure 10.2) When I questioned the merit of analog current faulting on ECU rev 3.0, this was Alex Ezzat's advice.

In industry - where you have to prove to the NHTSA that your 800 V battery pack is safe enough for anyone to drive around - layers of redundancy are an extremely important consideration for safety-critical systems. In a worst case, an over current fault could mean your battery's positive and negative terminals are shorted, potentially an extremely dangerous situation.

In the quest for achieving industry-level design on UBC Solar, we implemented 4 methods of over current faulting on Brightside:

1. Firmware-driven (ADC Reading)
2. Firmware-driven (Analog Watchdog)
3. Hardware-driven (Hardware current fault circuitry)
4. Hardware-driven (Main pack fuse)

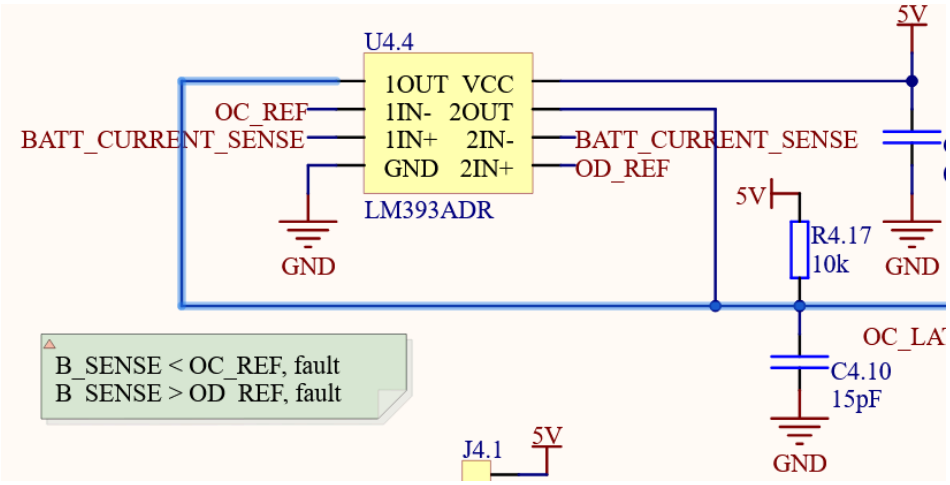
Two of our over current methods rely on our STM32 microcontroller working. In one case, we're in the main loop of the microcontroller and comparing a read current value to the allowable range (over charge limit to over discharge limit, -20 to 60 A). The other method is using the Analog Watchdog (AWDG) peripheral on the STM32 to monitor the exact ADC value we read. Because the AWDG operates outside of the main loop of our firmware, it will trigger a current fault even if our state machine is stuck somewhere. This eliminates our firmware as a variable.

The final line of defense against current faults is our 80 A fuse. Seeing as this fuse costs $100 on Digikey with one in stock at the time of writing, we'd like it to be an absolute worst-case scenario with many other current fault methods before we reach it.

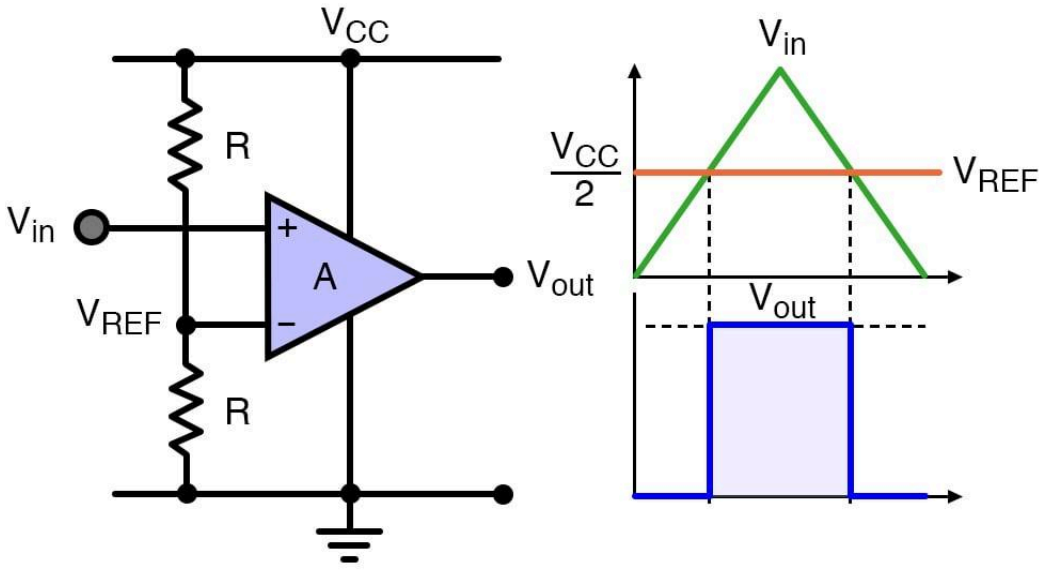
Hardware current fault circuitry is not dependent on the microcontroller and doesn't require any particularly expensive components that need to be replaced after use.

This circuitry reads the same analog current sensor voltage that the STM32 uses, then outputs a logic level high or low voltage to simulate ESTOP occurring, the same result as a firmware-driven current fault.

10.2 Comparator Logic



(Figure 10.3) This shows the LM393ADR two-channel comparator used to read the current sensor voltage output. This is two comparators in a single package.



(Figure 10.4) Comparator output visualization. When Vin > Vref, Vout goes high.

The first step to implementing hardware current faulting is using two comparators to sense whenever the current sensor voltage output goes out of the nominal range. We trigger a current fault if the output of either comparator ever goes low (0 V).

We have two fault currents:

1. Over charge (OC): -20 A. Charge current is current flowing into the pack, so this manifests as a more negative current sensor voltage output (V_Curr_Sense) relative to the 1.8 V reference (eg. 1.7 V could mean 10 A discharging)
2. Over discharge (OD): 60 A. Current flowing out of the pack results in a great voltage on V_Curr_Sense.

A standard rail-to-rail comparator has five 5 pins: source, ground, two inputs, and a single output. The two inputs are the inverting and non-inverting pins. If the inverting input (signified with +) has a higher voltage than the non-inverting input (signified with -), the output voltage is pulled to the source voltage. If the inverting input is lower voltage than the non-inverting input, output is pulled to ground.

We use the LM393ADR open-drain comparator which means it can only pull the comparator output to ground, it cannot pull the output to source voltage. Instead, if the inverting input is greater than the non-inverting input, the output is floating.

For the OC fault, a comparator's inverting output is hooked up to V_Curr_Sense, and the non-inverting output is the over current reference voltage (OC_Ref).

For the OD fault, a different comparator's inverting output is hooked up to the OD voltage reference (OD_Ref), and the non-inverting output is V_Curr_Sense.

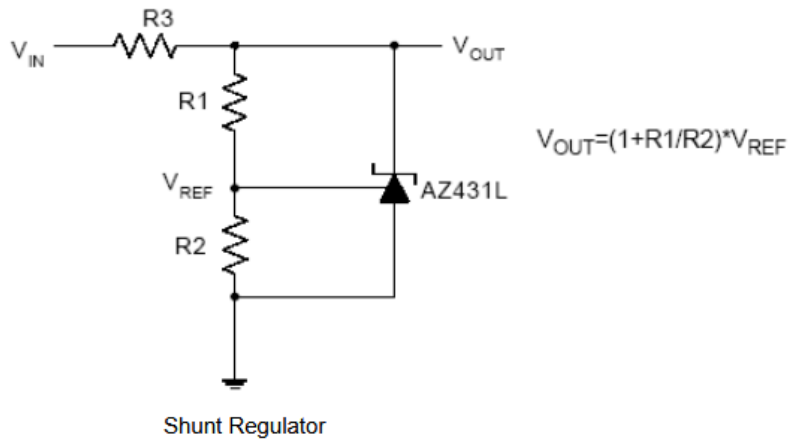
In the OC comparators default state (OC_Ref < V_Curr_Sense, ie. non-inverting < inverting), its output is floating.

In the OD comparators default state (V_Curr_Sense < OD_Ref, ie. non-inverting < inverting), its output is floating.

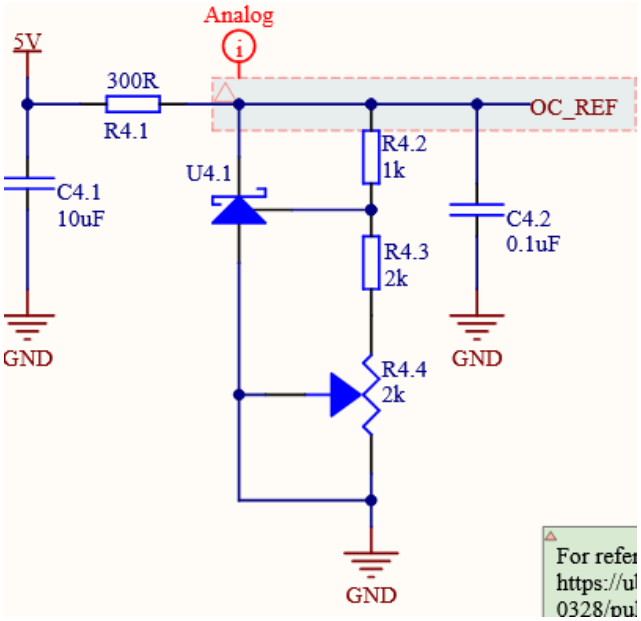
Note that both comparator's output to the same trace that is pulled up to 5 V with a 10k resistor. Because our comparators are open-drain, when we get into a fault state this trace is pulled to ground and is registered as a fault.

If we used a rail-to-rail comparator, the default state of our comparators would be 5 V instead of floating. Then, when one of their outputs is ground we would effectively be shorting 5 V and GND.

10.3 Generating Voltage References



(Figure 10.5) Taken from the AZ431LANTR-G1 datasheet, this is the circuit we use for our voltage reference.



(Figure 10.6) This is our implementation of the datasheet's typical application circuit.

For proper functioning of the comparator, we need to set the OC_Ref and OD_Ref voltage references. To achieve this, we use two AZ431LANTR-G1 shunt voltage reference ICs.

Figure 10.5 shows the formula used to determine the output voltage of these voltage references. By sizing the resistors R1 and R2 appropriately, we can make the output voltage any multiple of the reference voltage (1.24 V). We use a potentiometer to fine tune the equivalent resistance of R2. Note that the reference voltage of 1.24 V is generated internally by the IC.

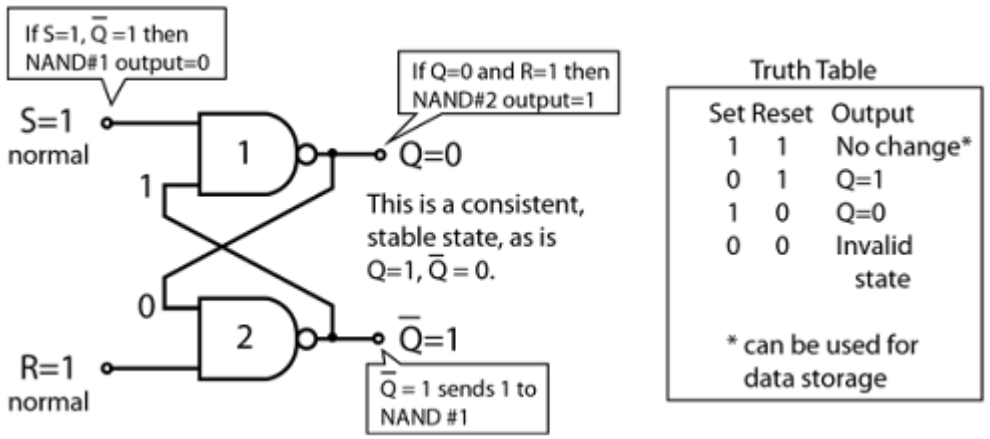
Notice that we use a 300 ohm resistor (R4.1) to provide the source voltage for the voltage reference circuit. A shunt voltage reference operates by maintaining a fixed voltage across its terminals, shunting all excess current through itself.

Ohm’s law states that V = IR. In other words, the voltage drop over a resistor is proportional to the current flowing through it. The voltage drop over R4.1 is equal to 5 V - OC_Ref. Given a resistance of 300 ohms, we can use V = IR to find the current flowing through the resistor.

Since OC_Ref is a voltage reference and has negligible current draw, all of the current flowing through the resistor R4.1 either flows through the voltage reference IC or the voltage divider (R4.2, R4.3, R4.4). Because the voltage divider’s equivalent resistance is far higher than the voltage reference, most of the current flows through the voltage reference.

This is the principle of operation of a voltage reference, the voltage drop over R4.1 is how the reference voltage is created. That voltage drop is proportional to the constant voltage over shunt voltage reference IC, which is controllable through a voltage divider.

10.4 NAND Latch Logic



(Figure 10.7) NAND latch diagram. Notice the truth table. In our implementation, S is always held at high (1) and if R ever goes low (0), the NAND latch output will be 0 until the circuit is power cycled.

Now that from the comparators we have a trace that goes low whenever we’re in a current fault, we need a way of remembering that a fault occurred. The intent with all our battery faults is that we open all contactors and isolate the pack whenever any fault occurs. Hence, the only way to recover from a fault is to power cycle the battery pack. If we don’t have a way of remembering that a current fault occurred (even if just for a millisecond), then the pack will return to an operational state, which is not desired.

We use a CMOS NAND Latch to remember if a current fault ever occurred. CMOS stands for Complementary Metal Oxide Semiconductor, and is a technology that is used to store information while requiring very low power draw. The “complementary” in CMOS refers to its utilization of both P-channel and N-channel MOSFETs. Because FETs require defined gate voltages to remain in a given state, CMOS circuits do not retain information after power is cut off. This makes it ideal for our circuit, where we want to remember a fault occurred but only for the current power cycle.

Figure 10.7 shows a schematic for a NAND latch and its truth table. Notice whenever Set is high and Reset is low, we store a 0. Whenever Set is low and Reset is high, we store a 1. If both are high, the stored value is whatever was previously set. The value of Q is what is outputted from our CMOS NAND Latch’s.

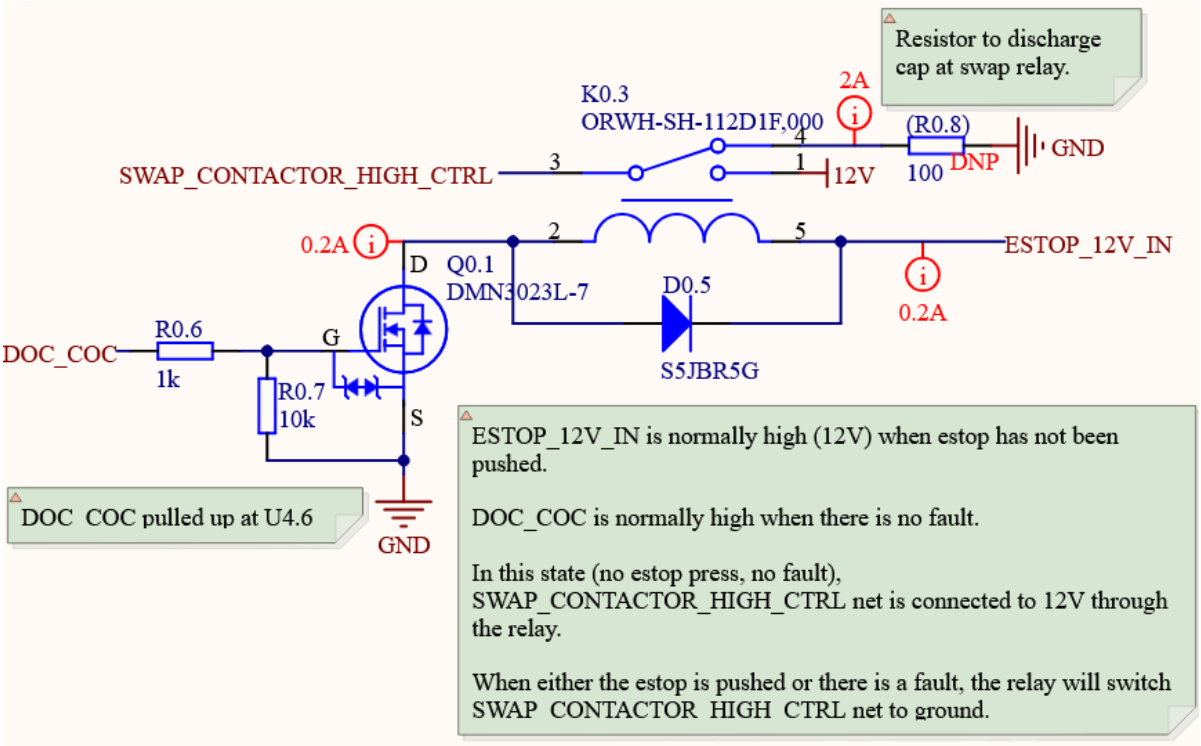
In our implementation, we use a GPIO to always drive the Set pin high and Reset is the output of the comparators.

During ECU startup, the Set GPIO pin isn’t set high instantly but the Reset pin is already high from the comparator circuit. This stores a 1 in the output state.

Next, the Set GPIO pin is pulled high by the microcontroller and the NAND Latch goes into its memory state, where it continues outputting a 1.

Finally, if the reset pin ever goes low (signifying a current fault), the output state becomes 0. If the reset pin is ever pulled high again, we are brought back into the memory state where we maintain the 0 output.

10.5 Simulating ESTOP



(Figure 10.8) DOC_COC is the output of the NAND Latch. When it goes low, the Swap / Contactor High Control Relay opens, hence all contactor coils lose their voltage source and open. See sections 8.2 or 1.1 for more info.

Now that we have a trace that is pulled to ground and remains low if we ever have a current fault, we need a way for this to isolate the battery pack.

Because this trace isolates the battery pack if we ever have an over charge or over discharge event, we call it The Discharge Over-Current or Charge Over-Current (DOC_COC) trace.

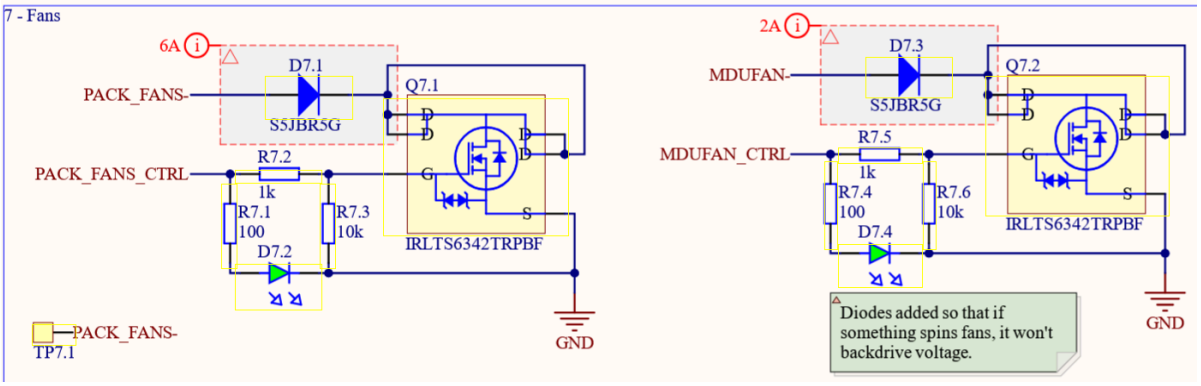
From section 8.2 we already know we have a method of isolating the battery pack if a voltage ever goes low. When ESTOP is pressed, the ESTOP_12V_IN trace on the ECU goes low and the Swap / Contactor High Control Relay opens, and hence all contactors open.

We put an N-channel MOSFET controlled by DOC_COC in series with the ESTOP_12V_IN line so that if DOC_COC ever goes low, there is no path for current to flow through the Swap / Contactor high Control Relay's coils and it opens. This way, we simulate ESTOP being pressed and open all contactors.

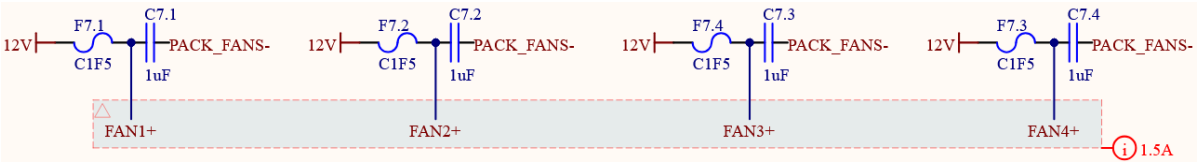
[Why didn't it work with ECU rev 2.0? And why didn't we try to fix it between comp 2024 and 2025?]

[Why a 4 channel CMOS IC? We only use one]

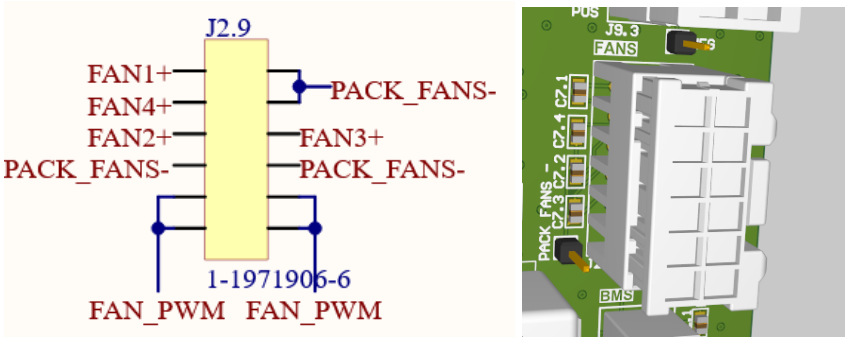
11. Pack Fans Power



(Figure 11.1) The shared ground of all fans is toggled by a low-side NFET. Note the diode to prevent reverse current.



(Figure 11.2) Each of the four fans has a 5 A fuse and 1 uF parallel capacitor.



(Figure 11.3) These two images show the pack fans connector, with separate power, ground, and PWM pins.

The ECU has provisions to provide power to both the pack fans and the motor controller fans. Note that with the Mitsuba Motor Controller we use on Brightside we don’t need to power the motor controller fan ourselves, this is a function of the motor controller.

Low-side switching is done with a IRLTS6342TRPBF N-channel power mosfet rated for 8.3 A. This toggles the ground of all of the pack fans, all pack fan current goes through this FET. Meanwhile, on the high-side, each fan has its own individual 5 A fuse and a 1uF bulk capacitor.

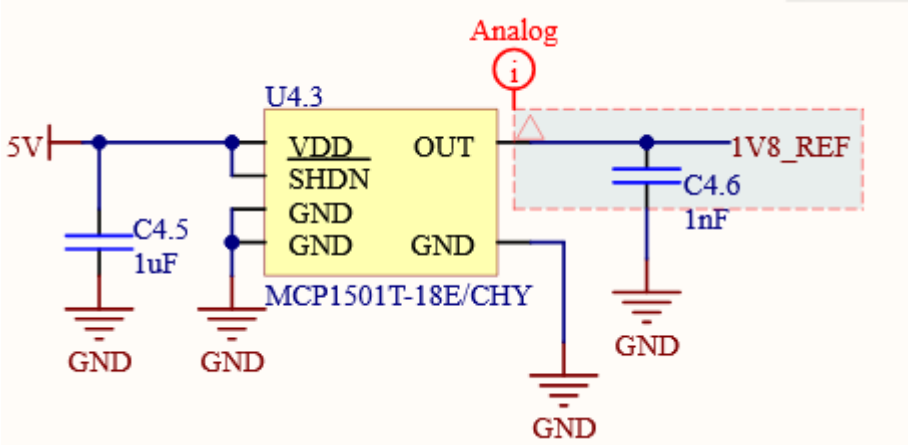
The fan PWM signal is generated by the masterboard, wired to the ECU, then given to the fans. This is a result of basing our BMS architecture off the Elithion off-the-shelf BMS as mentioned in section 2.1.

Because an electric motor running in reverse acts as a generator, we need to prevent inadvertent spinning of the fans from generating reverse current. This is done with a series diode.

Because a generator's voltage is proportional to rpm (eg. 0.01 V / rpm), the voltage of the reverse-current will not be 12 V. In a worst case scenario, the battery may be off and spinning of the fans may cause a voltage spike or voltage higher than can be tolerated, damaging components or inadvertently turning on ECU circuitry.

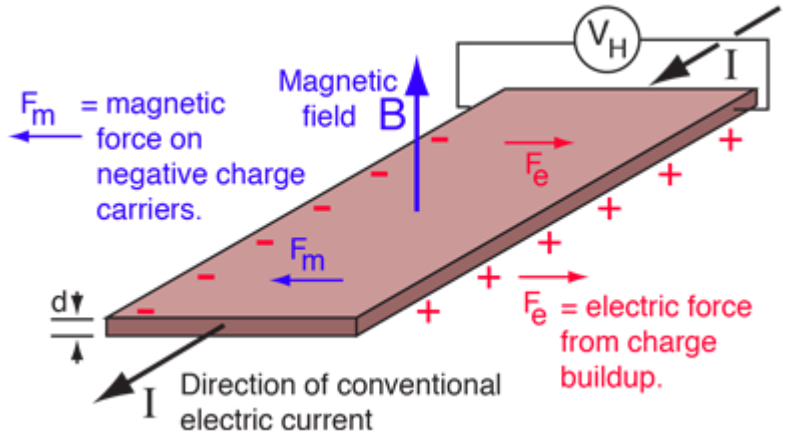
[Why was a flyback diode not used here? You have to deal with the forward voltage drop of the diode. This same series diode was used on ECU rev 1.0]

12. Main Pack Current Sensing



(Figure 12.1) Voltage Reference IC, providing a stable 1.8 V.

(Figure 12.2) Our HASS-100S Current Sensor during testing with 30 loops of wire around it, to simulate reading 30x current. Our PSUs only go up to 3 A, but our pack goes up to 60 A.



(Figure 12.3) Diagram explaining the Hall Effect.

We use a HASS-100S current sensor in Brightside's pack. This current sensor leverages the Hall Effect to produce a voltage difference that is proportional to current flowing through a conductor.

Because a flowing current produces a magnetic field proportional to that current, if we can sense that magnetic field we can derive current. A magnetic field exerts a force on electrons which manifests as a voltage drop in the axis orthogonal to both the magnetic field and current flow (shown in figure 12.3). Our STM32 microcontrollers can sense voltage, so we read this voltage and use a formula from the datasheet to derive current.

Because our battery pack can be charged and discharged, we need a way to sense negative current. If the hall effect voltage is in reference to ground, then charging the pack would result in a negative voltage, which our microcontrollers cannot read. So, the HASS-100S has a pin which allows us to set a reference voltage. We use a fixed 1.8 V output voltage reference IC for this purpose (Figure 12.1).